Sistema de Gestión de Salas para Espacios de Coworking

A.B. Keylin Pamela, Estudiante Universitario, UFidélitas; C.E. Raquel Francini, Estudiante Universitario, UFidélitas, G.A. Jefry, Estudiante Universitario, UFidélitas, M.G. Flor De Liz, Estudiante Universitario, UFidélitas y P.C. Nathalie, Estudiante Universitario, UFidélitas (Universidad Fidélitas, San José-San Pedro, kaguirre00773@fide.ac.cr, rcabezas90858@ufide.ac.cr, jgoleath60816@ufide.ac.cr, fmartinez00873@ufide.ac.cr, npinar00849@ufide.ac.cr

Resumen

Este proyecto presenta el diseño de un sistema de gestión de salas para un espacio de coworking, con el fin de organizar las reservas y evitar conflictos en el uso de salas compartidas. El sistema se modela mediante una base de datos relacional, tomando en cuenta conceptos como entidades, relaciones, integridad referencial y restricciones para evitar duplicación de horarios.

El desarrollo del sistema se basa en el análisis de requerimientos, la elaboración de un diagrama entidad–relación y la transformación del modelo conceptual a un modelo relacional. Como resultado, se obtiene una estructura de base de datos simple, funcional y escalable que permite registrar usuarios, salas, reservas y cambios realizados por los usuarios del coworking. Este proyecto aplica los conocimientos del curso de Lenguaje de Bases de Datos en un contexto práctico y cercano a situaciones reales.

Palabras clave: *Coworking, bases de datos relacionales, reservas de salas, sistema de gestión, Diagramas.*

1 INTRODUCCIÓN

La organización de espacios compartidos representa un reto cuando varias personas necesitan utilizar los mismos recursos en distintos horarios, por lo que el modelo de coworking se ha vuelto cada vez más común como alternativa a las oficinas tradicionales, especialmente para trabajadores independientes, pequeñas empresas y equipos remotos. Estos espacios ofrecen salas de reuniones y áreas compartidas que deben administrarse correctamente para evitar conflictos de uso y pérdida de tiempo, por lo que se desarrollará un sistema en el cual se podrán gestionar las reservas de las salas, haciendo que sea más

fácil para los usuarios de estas salas, y evitar conflictos.

De manera que, enfocándolo a un punto de vista académico, en este proyecto se podrán aplicar conceptos vistos en el curso de bases de datos como entidades, relaciones, restricciones e integridad de datos, permitiendo solucionar un problema de la vida real, gracias a la gestión y aprendizaje adquirido en base de datos.

2 ANTECEDENTES

En la última década, los espacios de coworking han experimentado un crecimiento significativo debido al aumento del trabajo remoto, el emprendimiento y la necesidad de ambientes laborales flexibles. Estos espacios ofrecen salas de reuniones, oficinas privadas y áreas colaborativas que requieren una gestión organizada para evitar conflictos de horarios y optimizar el uso de los recursos disponibles. La administración manual de reservas mediante agendas físicas o archivos compartidos puede generar errores, duplicación de horarios y falta de control sobre la disponibilidad de las salas.

Ante esta situación, el uso de sistemas informáticos y bases de datos relacionales se ha convertido en una solución eficiente para gestionar reservas, usuarios y recursos dentro de los espacios de coworking. Estos sistemas permiten centralizar la información, automatizar procesos y mejorar la toma de decisiones relacionadas con la ocupación de las salas. De acuerdo con Silberschatz et al. (2020), las bases de datos relacionales facilitan la organización estructurada de la información y permiten mantener la integridad y consistencia de los datos en sistemas de gestión. Por ello, el desarrollo de un sistema de gestión de salas basado en bases de datos representa una herramienta fundamental para mejorar la administración de los espacios de trabajo compartido.

2.1 Objetivo general

Proponer el diseño de un sistema informático que facilite la administración del uso de salas en un espacio de coworking, mediante una estructura de base de datos que permita organizar la información de forma eficiente.

2.2 Objetivos específicos

- Identificar las necesidades principales relacionadas con el uso y control de salas dentro de un espacio de coworking.

- Establecer los elementos de información que deben almacenarse

para lograr una gestión ordenada de los espacios disponibles.

- Desarrollar una propuesta de solución basada en bases de datos que mejore la organización y el control del uso de las salas.

2.3 Trabajos relacionados

Existen diversas soluciones tecnológicas orientadas a la gestión de reservas y administración de espacios de trabajo. En primer lugar, los sistemas de reservas en línea permiten visualizar la disponibilidad de recursos en tiempo real y evitar conflictos de horarios, lo que mejora la organización interna de las empresas (Laudon & Laudon, 2021).

En segundo lugar, los sistemas de gestión de coworking integran funciones como registro de usuarios, control de salas y seguimiento de reservas, demostrando la importancia de contar con una base de datos centralizada que relacione la información de los diferentes elementos del sistema (Nexudus, 2023).

Finalmente, el modelado entidad–relación es una técnica ampliamente utilizada en el diseño de bases de datos para representar de manera gráfica las relaciones entre entidades como usuarios, salas y reservas. Esta metodología facilita la comprensión del sistema y permite construir estructuras de datos eficientes y escalables (Coronel & Morris, 2019). Estos trabajos sirven como base conceptual para el desarrollo del presente proyecto académico.

3 METODOLOGÍA

3.1 Tipo de investigación

En esta investigación, el enfoque es aplicado, ya que su finalidad es desarrollar una solución tecnológica que resuelva un problema real de gestión de reservas en espacios de coworking. Este tipo de investigación busca usar conocimientos teóricos para generar herramientas prácticas que mejoren procesos existentes. Según el Instituto Tecnológico de Costa Rica, la investigación aplicada se enfoca en resolver problemas específicos mediante el uso de metodologías técnicas y científicas (TEC, 2023).

Asimismo, se utiliza un enfoque *descriptivo*, puesto que se analizan y detallan los procesos actuales de administración de salas, identificando cómo se gestionan las reservas, qué información es necesaria y qué datos deben almacenarse. La investigación

descriptiva permite caracterizar un fenómeno y documentar sus elementos clave de manera sistemática (Universidad de Valencia, 2022).

3.2 Descripción de los instrumentos

Para el desarrollo del proyecto se utilizaron instrumentos básicos y accesibles que permiten recopilar información y organizar los datos necesarios para el sistema, sin complicar su aplicacion:

a) **Revisión de información en línea**
Se consultaron recursos sencillos en internet sobre gestión de reservas, organización de espacios de trabajo y conceptos básicos de bases de datos. Este tipo de revisión permite obtener una comprensión general del tema de manera rápida y accesible. Google destaca que la investigación en línea facilita encontrar información actualizada y relevante para proyectos académicos (Google, 2024).
https://support.google.com/websearch/answer/4540094

b) **Observación de procesos reales**
Se realizó una observación del proceso típico que sigue una persona cuando reserva una sala en un coworking. Este instrumento permite identificar qué datos se necesitan y qué pasos deben registrarse. Según Notion, la observación de flujos de trabajo ayuda a detectar qué información debe organizarse o estructurarse en un sistema digital (Notion, 2023).
https://www.notion.so/help

c) **Uso de herramientas digitales para diagramas simples**
Para visualizar las ideas y entender mejor la estructura del sistema, se utilizaron herramientas básicas como diagramas en línea (por ejemplo, Canva). Canva señala que los diagramas ayudan a representar procesos, elementos e información de manera clara antes de construir un sistema (Canva, 2024).
https://www.canva.com/es_mx/diagramas/

d) **Consulta de ejemplos de bases de datos en línea**
Se revisaron ejemplos de modelos de bases de datos disponibles en plataformas educativas, lo que permitió comprender diferentes

formas de estructurar entidades y relaciones. Sitios como Lucidchart ofrecen ejemplos de diagramas entidad–relación que ayudan a visualizar cómo se organizan datos en sistemas reales (Lucidchart, 2024).
https://www.lucidchart.com/pages/examples/er-diagram-examples

e) **Guías y documentación técnica básica**

Se consultaron documentación accesible sobre principios de diseño de bases de datos, disponible gratuitamente en Microsoft Learn. Estas guías ayudan a definir llaves primarias, relaciones y reglas de integridad de manera clara para principiantes (Microsoft Learn, 2024), entendiendo su proposito y aplicandolo en el proyecto.
https://learn.microsoft.com/en-us/office/troubleshoot/access/database-design-basics

f) **Análisis de requisitos mediante listas y esquemas**

Tambien se elaboraron listas simples para organizar los requerimientos funcionales y no funcionales del sistema. Según IBM, el uso de listas y esquemas facilita ordenar ideas y asegurar que no se omitan necesidades del usuario durante el diseño del sistema (IBM, 2023).
https://www.ibm.com/topics/requirements-gathering

g) **Revisión de plataformas de coworking existentes**

Un punto muy importante es conocer el mercado que se abordara, por eso se analizaron páginas web de plataformas reales de coworking, como Nexudus, para observar qué información registran en un sistema de reservas y qué funciones suelen incluir. Esto permite comparar el proyecto con soluciones reales del mercado y asi poder ofrecer soluciones factibles (Nexudus, 2024).

https://www.nexudus.com

3.3 Estrategia de desarrollo: Lenguajes a utilizar y entornos de desarrollo

Para desarrollar el sistema de gestión de salas se planteó una estrategia práctica y acorde con los objetivos del proyecto. La idea es empezar por

organizar la información, luego diseñar la estructura de la base de datos y finalmente implementarla en un entorno que sea fácil de utilizar y probar. Esta estrategia combina herramientas accesibles y de uso gratuito, con lenguajes que se van a conocer durante el curso.

3.3.1 Lenguajes a utilizar

El lenguaje principal que se utilizará es Oracle SQL Developer, ya que es el estándar para trabajar con bases de datos relacionales. Con Oracle SQL Developer es posible crear tablas, establecer relaciones y asegurar la integridad de los datos. Además, Oracle SQL Developer es recomendado para proyectos donde se necesita manejar información estructurada y realizar consultas, como en el caso de un sistema de reservas (Microsoft Learn, 2024).

3.3.2 Herramientas y entornos de desarrollo

Para crear el diagrama entidad–relación y visualizar la estructura del sistema se utilizará MySQL Workbench, ya que es una herramienta gráfica que permite modelar bases de datos de manera visual y, además, facilita generar posteriormente el script SQL de forma automática. Workbench también ayuda a validar las relaciones y detectar errores en el diseño antes de ejecutar cualquier consulta (MySQL, 2024).

De forma complementaria, también pueden utilizarse herramientas de apoyo visual como Lucidchart o Canva en caso de que se requiera hacer bocetos previos o representar interacciones del sistema. Estas herramientas permiten construir diagramas de forma intuitiva y ayudan a comprender la estructura general del proyecto (Lucidchart, 2023).

En la parte de implementación se utilizará *MySQL*, debido a que es uno de los gestores de bases de datos más utilizados y porque cuenta con documentación accesible y una curva de aprendizaje amigable para estudiantes. MySQL permite crear la base de datos, ejecutar consultas y validar que las relaciones estén funcionando correctamente (MySQL, 2024), además porque es uno de los requerimientos establecidos para la ejecución del presente trabajo.

El uso de MySQL Workbench permitió definir explícitamente los tipos de datos, claves primarias y claves foráneas, validando la integridad referencial antes de generar el script SQL. Esta validación temprana del diseño reduce errores estructurales y mejora la calidad del sistema final (IBM, 2023).

3.3.3 El desarrollo del sistema seguirá tres pasos principales:

a) Modelado conceptual:

Se identificarán las entidades, atributos y relaciones del sistema, y se construirá el diagrama ER directamente en MySQL Workbench utilizando su módulo de *EER Diagram*.

b) Transformación al modelo relacional:

Una vez validado el diagrama, se definirá el modelo relacional incorporando claves primarias, foráneas y reglas de integridad. Workbench permite generar automáticamente el esquema SQL, lo cual facilita esta etapa.

c) Implementación y pruebas:

Finalmente, se crearán las tablas en Oracle SQL Developer y se realizarán pruebas básicas con datos de ejemplo, para confirmar que la base de datos funciona correctamente. IBM señala que la validación temprana del diseño reduce errores y mejora la calidad del sistema final (IBM, 2023**).**

4 ANÁLISIS DE RESULTADOS

En esta sección se presentan los principales resultados obtenidos durante el desarrollo del proyecto hasta la fecha. A partir del análisis de los requerimientos del sistema y del estudio del proceso de reservas en espacios de coworking, se definieron las entidades necesarias y su estructura dentro de la base de datos. Además, se elaboró el diagrama entidad–relación (ER), el cual permite visualizar de manera clara cómo se organiza la información y cómo se relacionan los distintos elementos del sistema. Estos resultados representan la base del funcionamiento del sistema y aseguran que los datos puedan gestionarse de forma ordenada, consistente y sin conflictos de horarios.

4.1 Requerimientos del proyecto

Se diseñaron un total de 12 tablas, entre las cuales destacan:

- USUARIOS: almacena la información de los usuarios del sistema.

- SALAS: contiene los datos de las salas disponibles.
- RESERVAS: registra las reservas realizadas.
- ESTADOS: define el estado de cada reserva.
- PAGOS: almacena la información de pagos asociados.
- SERVICIOS y RESERVA_SERVICIOS: permiten agregar servicios adicionales.
- BITACORA_CAMBIOS: guarda el historial de modificaciones.

Estas estructuras fueron diseñadas siguiendo principios de integridad referencial mediante el uso de:

Llaves primarias

Llaves foráneas

Restricciones (UNIQUE, NOT NULL)

Esto permite garantizar que los datos sean consistentes, evitando errores como reservas duplicadas o registros inválidos

El diccionario de datos completo se encuentra en los anexos (Anexo 22 a 23), donde se detallan atributos, tipos de datos y restricciones de cada tabla.

4.1.1 Requerimientos funcionales

a) El sistema debe permitir registrar usuarios del coworking con información básica como nombre, correo electrónico y tipo de usuario.

b) El sistema debe permitir realizar reservas de salas indicando fecha, hora de inicio y hora de finalización, evitando la duplicación de reservas en un mismo rango de tiempo mediante validaciones sobre los intervalos registrados.

c) El sistema debe permitir cancelar y modificar reservas existentes, dejando registrado el historial de cambios realizados y la fecha en que se hicieron.

d) El sistema debe generar un listado diario de ocupación de salas, donde se pueda ver qué espacios están reservados, en qué horarios y por qué usuario

e) El sistema debe permitir asignar prioridades de uso a las salas según el tipo de actividad (reuniones, capacitaciones u otros eventos).

f) El sistema debe permitir generar alertas cuando una sala esté próxima a finalizar su tiempo de uso.

4.1.2 Requerimientos no funcionales

- El sistema debe garantizar la integridad y seguridad de la

información almacenada en la base de datos mediante controles de acceso y validación de datos.

- El sistema debe presentar una interfaz sencilla que permita consultar la disponibilidad de las salas de manera rápida y eficiente.
- El sistema debe contar con control de permisos por tipo de usuario (por ejemplo: administrador y cliente), limitando qué acciones puede realizar cada uno dentro del sistema.
- El diseño de la base de datos debe ser escalable, de manera que permita agregar nuevas sedes o más salas en el futuro sin necesidad de rediseñar toda la estructura.
- El sistema debe contar con una estructura de datos clara y ordenada que facilite su mantenimiento y futuras mejoras.
- El sistema debe mantener un nivel adecuado de confiabilidad, asegurando que la información almacenada no se pierda ante errores comunes del sistema.

- Las contraseñas de los usuarios se almacenarán mediante técnicas de hash (CONTRASENA_HASH), evitando el almacenamiento de claves en texto plano y fortaleciendo la seguridad del sistema. De acuerdo con Oracle (2023), la protección de credenciales mediante cifrado o hash es una práctica esencial para garantizar la seguridad en bases de datos relacionales.

- **Diagrama modelo relacional**

El diagrama entidad–relación (ER) representa la estructura principal del sistema de gestión de salas para espacios de coworking. A través de este diseño se identifican las entidades fundamentales del sistema, así como las relaciones que existen entre ellas. El objetivo de un diagrama es visualizar de forma clara cómo se organiza la información relacionada con usuarios, salas, tipos de sala y reservas, permitiendo asegurar que la base de datos funcione de manera ordenada y sin conflictos de horarios.

El ER incluye entidades como *usuarios*, *salas*, *reservas*, *roles*, *tipo de sala* y una *bitácora de cambios*. Estas entidades permiten registrar no solo las reservas realizadas, sino también los roles de los usuarios y el historial de modificaciones dentro del sistema. Con esta estructura, se busca garantizar la integridad de los

datos, evitar duplicaciones y facilitar el control de las salas disponibles. En conjunto, el diagrama proporciona la base conceptual necesaria para implementar el sistema en Oracle SQL Developer y asegurar un funcionamiento eficiente.

- **Descripción del modelo relacional**

El diagrama entidad–relación del proyecto muestra la estructura completa del sistema de gestión de salas para espacios de coworking. Este diseño incluye 12 tablas, cumpliendo con el requisito del curso, y representa cómo se relacionan los diferentes componentes necesarios para administrar salas, usuarios, reservas, pagos y servicios adicionales.

A continuación, se describe la función de cada entidad y sus relaciones:

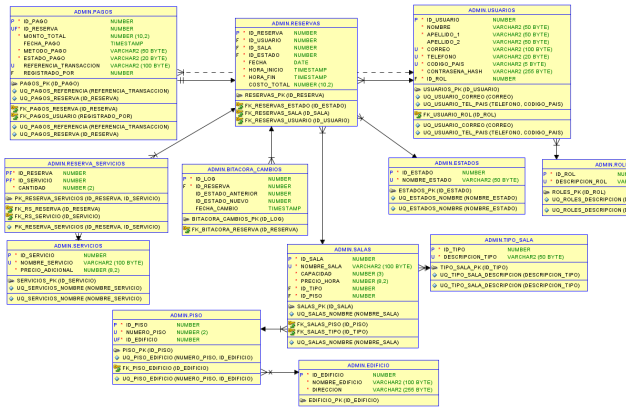


Figura 1, *Diagrama ER*

a) **Edificio:**

Representa la sede o ubicación física donde existen las salas.

Incluye información como nombre del edificio y dirección.
Un edificio puede tener varios pisos, por lo que se relaciona con la tabla piso en una relación 1 a muchos.

b) Piso

Contiene los números de piso dentro de un edificio.
Cada piso está asociado a un único edificio.
A su vez, un piso puede tener varias salas, por lo que se relaciona con salas.

c) Salas

Almacena los datos de cada sala disponible en el coworking (nombre, capacidad, tipo de sala y su piso correspondiente).
Está relacionada con:

- tipo_sala (qué tipo de sala es)
- piso (en qué piso está ubicada)
- reservas (las reservas asociadas a esa sala)

Una sala puede tener muchas reservas a lo largo del tiempo.

d) Tipo_sala

Define la categoría de cada sala (por ejemplo, sala de reuniones, sala privada, sala de capacitación).
Cada tipo puede aplicarse a múltiples salas.

e) Usuarios

Registra a los usuarios del coworking que pueden crear reservas.
Incluye nombre, correo electrónico y el rol del usuario.
Un usuario puede tener múltiples reservas.

f) Roles

Define los tipos de usuario en el sistema (administrador, cliente, recepción, etc.).
Cada usuario pertenece a un solo rol.

g) Reservas

Es una de las tablas centrales del sistema. Almacena toda la información relacionada con la reserva de una sala, incluyendo:

- *Usuario que realiza la reserva*
- *Sala reservada*
- *Estado actual de la reserva*
- *Fecha de la reserva*
- *Hora de inicio (HORA_INICIO)*
- *Hora de finalización (HORA_FIN)*
- *Costo total (COSTO_TOTAL)*

La duración de la reserva se obtiene como la diferencia entre HORA_FIN y HORA_INICIO, evitando almacenar datos redundantes y asegurando mayor consistencia en la información. Según Coronel y Morris (2019), evitar atributos derivados innecesarios mejora la integridad del modelo relacional y reduce inconsistencias.
El campo COSTO_TOTAL representa el monto final de la reserva. Este se calcula considerando el precio por hora definido en la tabla SALAS y los servicios adicionales asociados mediante la tabla RESERVA_SERVICIOS. Este enfoque sigue buenas prácticas de diseño de bases de datos orientadas a mantener coherencia y trazabilidad financiera (Oracle, 2023).

h) Estados

Indica el estado actual de una reserva (pendiente, confirmada, cancelada, completada, etc.).

Una reserva solo puede tener un estado a la vez, pero los cambios quedan registrados en la bitácora.

i) Bitacora_cambios

Registra cada modificación realizada sobre una reserva.
Incluye la fecha del cambio y referencias al estado anterior y al nuevo estado mediante claves foráneas hacia la tabla ESTADOS, garantizando integridad referencial.

Este diseño permite mantener un historial estructurado de cambios, asegurando consistencia de datos y trazabilidad de las transiciones de estado, lo cual es recomendado en sistemas transaccionales (Pressman & Maxim, 2020).

j) Servicios

Son los servicios adicionales que pueden complementarse con una reserva, como café, proyectores, catering o equipo audiovisual.
Incluyen un precio adicional.

k) Reserva_servicios

Tabla intermedia que permite asociar varios servicios a una misma reserva.
Es una relación de muchos a muchos entre reservas y servicios.
Además, registra la cantidad de cada servicio solicitado.

l) Pagos

Registra los pagos realizados por una reserva, incluyendo:

- monto total
- fecha del pago
- método de pago

Cada reserva puede tener un pago asociado, dependiendo del modelo del coworking.
Con esta primera etapa se definió claramente el objetivo del proyecto y la necesidad de un sistema que gestione de forma ordenada las reservas en un espacio de coworking. A partir del análisis de antecedentes, requerimientos y procesos reales, se establecieron las entidades principales y la estructura de datos necesaria para el sistema.
Además, se seleccionaron los instrumentos y herramientas de trabajo, y se definió la estrategia de desarrollo que guiará las siguientes fases. El diagrama entidad–relación elaborado en MySQL Workbench resume la organización del sistema y sirve como base para el diseño relacional. Con esto, el proyecto queda

preparado para avanzar hacia la implementación técnica

4.2 Arquitectura y fases futuras del sistema

En la presente etapa se desarrolló el diseño estructural y relacional de la base de datos como núcleo central del sistema. No obstante, el proyecto contempla su expansión hacia una arquitectura de tres capas:

- ***Capa de datos:*** Base de datos relacional implementada en Oracle SQL Developer.
- *Capa lógica:* Procedimientos almacenados, funciones, triggers y validaciones implementadas en SQL y PL/SQL.
- ***Capa de presentación:*** Aplicación con interfaz gráfica que permitirá a los usuarios gestionar reservas, consultar disponibilidad y administrar pagos.

Esta estructura sigue el modelo de separación de responsabilidades recomendado en ingeniería de software, el cual facilita escalabilidad, mantenimiento y seguridad del sistema (Pressman & Maxim, 2020).

En fases posteriores se implementará la conexión entre la base de datos y una aplicación desarrollada en un lenguaje de programación adecuado, permitiendo operaciones CRUD completas sobre las tablas del sistema y automatización de procesos.

4.3 Seguridad y control de acceso.

El sistema implementa un modelo de control de acceso basado en roles, permitiendo restringir operaciones según el tipo de usuario (administrador, cliente, recepción). Asimismo, se emplean mecanismos de validación y almacenamiento seguro de credenciales mediante hash. Este enfoque contribuye a garantizar confidencialidad, integridad y disponibilidad de la información (Laudon & Laudon, 2021).\

Con el desarrollo del modelo relacional, la definición de reglas de integridad y la proyección hacia una arquitectura completa con capa lógica e interfaz gráfica, el proyecto establecerá una base sólida para la implementación integral del sistema de gestión de salas. La estructura diseñada cumplirá con principios formales de modelado relacional y buenas prácticas de ingeniería de

software, asegurando escalabilidad, seguridad y consistencia de datos.

4.4 Implementación de la base de datos

Una vez definido el modelo relacional, se procedió a la implementación de la base de datos utilizando Oracle Database. En esta etapa se desarrolló el script SQL mediante sentencias DDL, lo que permitió la creación de las tablas, la definición de atributos y la configuración de las relaciones entre ellas.

La base de datos fue implementada con un total de doce tablas, correspondientes a las entidades previamente definidas. Cada tabla fue construida respetando la estructura del modelo relacional, asegurando coherencia entre el diseño conceptual y su implementación práctica.

Hoja de Trabajo | Generador de Consultas

```oracle
CREATE TABLE PISO (
    ID_Piso      NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Numero_Piso NUMBER NOT NULL,
    ID_Edificio NUMBER NOT NULL,
    CONSTRAINT fk_piso_edificio FOREIGN KEY (ID_Edificio)
        REFERENCES EDIFICIO(ID_Edificio) ON DELETE CASCADE
);

-- 3. ROLES
CREATE TABLE ROLES (
    ID_Rol           NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Descripcion_Rol VARCHAR2(50) NOT NULL
);

-- 4. USUARIOS
-- 4. USUARIOS
-- 3. ROLES
CREATE TABLE ROLES (
    ID_Rol           NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Descripcion_Rol VARCHAR2(50) NOT NULL
);
```

Salida de Script | Tarea terminada en 0,425 segundos

```
Table EDIFICIO creado.

Table PISO creado.

Table ROLES creado.
```

```oracle
CREATE TABLE USUARIOS (
    ID_Usuario       NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Nombre           VARCHAR2(100) NOT NULL,
    Apellido_1       VARCHAR2(100) NOT NULL,
    Apellido_2       VARCHAR2(100) NOT NULL,
    Correo           VARCHAR2(100) UNIQUE NOT NULL,
    Telefono         VARCHAR2(20),
    Contrasena_Hash VARCHAR2(255) NOT NULL,
    ID_Rol           NUMBER,
    CONSTRAINT fk_usuario_rol
        FOREIGN KEY (ID_Rol) REFERENCES ROLES(ID_Rol) ON DELETE SET NULL
);

-- 5. TIPO_SALA
CREATE TABLE TIPO_SALA (
    ID_Tipo           NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Descripcion_Tipo VARCHAR2(50) NOT NULL
);

-- 6. SALAS
CREATE TABLE SALAS (
    ID_Sala      NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Nombre_Sala VARCHAR2(100) NOT NULL,
    Capacidad    NUMBER NOT NULL,
    Precio_Hora NUMBER(10,2) NOT NULL,
    ID_Tipo      NUMBER,
    ID_Piso      NUMBER,
    CONSTRAINT fk_sala_tipo FOREIGN KEY (ID_Tipo)
        REFERENCES TIPO_SALA(ID_Tipo) ON DELETE SET NULL,
    CONSTRAINT fk_sala_piso FOREIGN KEY (ID_Piso)
        REFERENCES PISO(ID_Piso) ON DELETE SET NULL
);
```

```oracle
-- 7. ESTADOS
CREATE TABLE ESTADOS (
    ID_Estado      NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Nombre_Estado VARCHAR2(50) NOT NULL
);

-- 8. RESERVAS
CREATE TABLE RESERVAS (
    ID_Reserva       NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    ID_Usuario       NUMBER NOT NULL,
    ID_Sala          NUMBER NOT NULL,
    ID_Estado        NUMBER NOT NULL,
    Fecha            DATE NOT NULL,
    Hora_Inicio      TIMESTAMP NOT NULL,
    Hora_Fin         TIMESTAMP NOT NULL,
    Costo_Total      NUMBER(12,2),
    CONSTRAINT fk_reserva_usuario FOREIGN KEY (ID_Usuario) REFERENCES USUARIOS(ID_Usuario),
    CONSTRAINT fk_reserva_sala FOREIGN KEY (ID_Sala) REFERENCES SALAS(ID_Sala),
    CONSTRAINT fk_reserva_estado FOREIGN KEY (ID_Estado) REFERENCES ESTADOS(ID_Estado)
);

-- 9. SERVICIOS
CREATE TABLE SERVICIOS (
    ID_Servicio      NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Nombre_Servicio VARCHAR2(100) NOT NULL,
    Precio_Adicional NUMBER(10,2) NOT NULL
);

-- 10. RESERVA_SERVICIOS
CREATE TABLE RESERVA_SERVICIOS (
    ID_Reserva  NUMBER NOT NULL,
    ID_Servicio NUMBER NOT NULL,
    Cantidad    NUMBER DEFAULT 1,
    PRIMARY KEY (ID_Reserva, ID_Servicio),
    CONSTRAINT fk_rs_reserva FOREIGN KEY (ID_Reserva) REFERENCES RESERVAS(ID_Reserva) ON DELETE CASCADE,
```

```
    CONSTRAINT fk_rs_reserva FOREIGN KEY (ID_Reserva) REFERENCES RESERVAS(ID_Reserva) ON DELETE CASCADE,
    CONSTRAINT fk_rs_servicio FOREIGN KEY (ID_Servicio) REFERENCES SERVICIOS(ID_Servicio)
);

-- 11. PAGOS
CREATE TABLE PAGOS (
    ID_Pago       NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    ID_Reserva    NUMBER NOT NULL,
    Monto_Total   NUMBER(10,2) NOT NULL,
    Fecha_Pago    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    Metodo_Pago   VARCHAR2(50),
    CONSTRAINT fk_pago_reserva FOREIGN KEY (ID_Reserva) REFERENCES RESERVAS(ID_Reserva)
);

-- 12. BITACORA_CAMBIOS
CREATE TABLE BITACORA_CAMBIOS (
    ID_Log            NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    ID_Reserva        NUMBER,
    ID_Estado_Anterior  NUMBER,
    ID_Estado_Nuevo   NUMBER,
    Fecha_Cambio      TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_bitacora_reserva
        FOREIGN KEY (ID_Reserva) REFERENCES RESERVAS(ID_Reserva) ON DELETE CASCADE,

    CONSTRAINT fk_bitacora_estado_ant
        FOREIGN KEY (ID_Estado_Anterior) REFERENCES ESTADOS(ID_Estado),

    CONSTRAINT fk_bitacora_estado_nue
        FOREIGN KEY (ID_Estado_Nuevo) REFERENCES ESTADOS(ID_Estado)
);
```

```
Salida de Script
Tarea terminada en 0,192 segundos

Table SALAS creado.

Table ESTADOS creado.

Table RESERVAS creado.

Table SERVICIOS creado.

Table RESERVA_SERVICIOS creado.

Table PAGOS creado.

Table BITACORA_CAMBIOS creado.
```

Figura 3. *Script de creación de tablas en Oracle Database.*

Durante esta fase se definieron llaves primarias para garantizar la identificación única de cada registro, así como llaves foráneas para establecer relaciones entre las tablas. Por ejemplo, la tabla RESERVAS mantiene referencias hacia USUARIOS, SALAS y ESTADOS, lo cual permite registrar información completa sobre cada reserva.

Adicionalmente, se implementaron restricciones de integridad, como la restricción UNIQUE en el atributo correo de la tabla USUARIOS, con el fin de evitar la duplicidad de registros. También se aplicaron reglas como ON DELETE CASCADE y ON DELETE SET NULL, las cuales permiten controlar el comportamiento de los datos en caso de eliminación de registros relacionados.

Estas prácticas son fundamentales en el diseño de bases de datos relacionales, ya que contribuyen a mantener la consistencia, confiabilidad y seguridad de la información almacenada (Oracle, 2023).

4.5 Poblamiento de la base de datos

Una vez implementada la estructura de la base de datos, se procedió al poblamiento mediante la inserción de datos de prueba. Este proceso permitió simular el funcionamiento real del sistema dentro de un entorno controlado.

Se registraron datos correspondientes a edificios, pisos, roles de usuario, usuarios, tipos de salas, salas, servicios adicionales, estados de reservas, reservas y pagos. Estos datos fueron diseñados para representar distintos escenarios de uso del sistema.

```sql
-- EDIFICIO
INSERT INTO EDIFICIO (Nombre_Edificio, Direccion)
VALUES ('Coworking Central', 'San Jose, Costa Rica');

INSERT INTO EDIFICIO (Nombre_Edificio, Direccion)
VALUES ('Coworking Escazu', 'Escazu, San Jose');

-- PISO
INSERT INTO PISO (Numero_Piso, ID_Edificio)
VALUES (1, 1);

INSERT INTO PISO (Numero_Piso, ID_Edificio)
VALUES (2, 1);

INSERT INTO PISO (Numero_Piso, ID_Edificio)
VALUES (1, 2);

-- ROLES
INSERT INTO ROLES (Descripcion_Rol)
VALUES ('Administrador');

INSERT INTO ROLES (Descripcion_Rol)
VALUES ('Cliente');

INSERT INTO ROLES (Descripcion_Rol)
```

```sql
-- USUARIOS
INSERT INTO USUARIOS (Nombre, Apellido_1, Apellido_2, Correo, Telefono, Contrasena_Hash, ID_Rol)
VALUES ('Flor', 'Martinez', 'Garcia', 'flor@correo.com', '8888-1111', 'hash123', 1);

INSERT INTO USUARIOS (Nombre, Apellido_1, Apellido_2, Correo, Telefono, Contrasena_Hash, ID_Rol)
VALUES ('Ana', 'Lopez', 'Sanchez', 'ana@correo.com', '8888-2222', 'hash456', 2);

INSERT INTO USUARIOS (Nombre, Apellido_1, Apellido_2, Correo, Telefono, Contrasena_Hash, ID_Rol)
VALUES ('Carlos', 'Perez', 'Mora', 'carlos@correo.com', '8888-3333', 'hash789', 3);

-- USUARIOS EXTRA (CORREGIDOS)
SELECT USER FROM dual;
INSERT INTO USUARIOS VALUES (DEFAULT, 'Alhana', 'Diaz', 'Aguirre', 'alhana@correo.com', '8888-4444', 'hash132', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Stephanie', 'Brenes', 'Araya', 'brenes@correo.com', '8888-5555', 'hash856', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Carlos', 'Gutierrez', 'Quiros', 'gutierrez@correo.com', '8888-6666', 'hash098', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Sandra', 'Rojas', 'Brenes', 'sandra@correo.com', '8877-1234', 'hash139', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Stephanie', 'Brenes', 'Dobles', 'dobles@correo.com', '8877-2345', 'hash176', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Jose', 'Hernandez', 'Perez', 'hernandez@correo.com', '8877-3456', 'hash086', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Martha', 'Fonseca', 'Artavia', 'martha@correo.com', '8877-4567', 'hash443', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Manuel', 'Cruz', 'Cordoba', 'cruz@correo.com', '8877-5678', 'hash406', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Alejandro', 'Leiva', 'Quiros', 'quiros@correo.com', '8877-5679', 'hash996', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Christian', 'Marin', 'Hernandez', 'christian@correo.com', '8877-6789', 'hash566', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Roberto', 'Brenes', 'Araya', 'roberto@correo.com', '8877-7890', 'hash444', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Marco', 'Perez', 'Ramirez', 'marco@correo.com', '8788-1122', 'hash098', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Emilce', 'Romero', 'Martinez', 'emilce@correo.com', '8788-2233', 'hash223', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'David', 'Araya', 'Figueres', 'david@correo.com', '8788-3344', 'hash445', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Marvin', 'Fernandez', 'Robles', 'marvin@correo.com', '8788-4455', 'hash454', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Kevin', 'Abarca', 'Coto', 'coto@correo.com', '8788-5566', 'hash109', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Naomy', 'Alvarado', 'Calderon', 'naomy@correo.com', '8788-6677', 'hash450', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Jimena', 'Mata', 'Barrantes', 'jimena@correo.com', '8788-7788', 'hash012', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Tamara', 'Ramos', 'Morales', 'tamara1@correo.com', '8788-8899', 'hash165', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Myriam', 'Aguirre', 'Chavarria', 'tamara2@correo.com', '8765-8899', 'hash165', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Catalina', 'Navarro', 'Marin', 'tamara3@correo.com', '8766-8899', 'hash887', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Patricia', 'Valencia', 'Mendoza', 'patri@correo.com', '8767-8899', 'hash889', 2);
```

```sql
-- RESERVA_SERVICIOS
INSERT INTO RESERVA_SERVICIOS (ID_Reserva, ID_Servicio, Cantidad)
VALUES (1, 1, 1);

INSERT INTO RESERVA_SERVICIOS (ID_Reserva, ID_Servicio, Cantidad)
VALUES (1, 2, 1);

INSERT INTO RESERVA_SERVICIOS (ID_Reserva, ID_Servicio, Cantidad)
VALUES (2, 3, 1);

-- PAGOS
INSERT INTO PAGOS (ID_Reserva, Monto_Total, Metodo_Pago)
VALUES (1, 43000, 'Tarjeta');

INSERT INTO PAGOS (ID_Reserva, Monto_Total, Metodo_Pago)
VALUES (2, 23000, 'SINPE');

-- BITACORA_CAMBIOS
INSERT INTO BITACORA_CAMBIOS (ID_Reserva, ID_Estado_Anterior, ID_Estado_Nuevo)
VALUES (1, 1, 2);

INSERT INTO BITACORA_CAMBIOS (ID_Reserva, ID_Estado_Anterior, ID_Estado_Nuevo)
VALUES (2, 1, 3);

COMMIT;
```

```sql
-- TIPO_SALA
INSERT INTO TIPO_SALA (Descripcion_Tipo)
VALUES ('Sala de reuniones');

INSERT INTO TIPO_SALA (Descripcion_Tipo)
VALUES ('Oficina privada');

INSERT INTO TIPO_SALA (Descripcion_Tipo)
VALUES ('Sala de capacitacion');

-- SALAS
INSERT INTO SALAS (Nombre_Sala, Capacidad, Precio_Hora, ID_Tipo, ID_Piso)
VALUES ('Sala A', 10, 15000, 1, 1);

INSERT INTO SALAS (Nombre_Sala, Capacidad, Precio_Hora, ID_Tipo, ID_Piso)
VALUES ('Sala B', 4, 10000, 2, 2);

INSERT INTO SALAS (Nombre_Sala, Capacidad, Precio_Hora, ID_Tipo, ID_Piso)
VALUES ('Sala C', 20, 25000, 3, 3);

-- ESTADOS
INSERT INTO ESTADOS (Nombre_Estado)
VALUES ('Pendiente');

INSERT INTO ESTADOS (Nombre_Estado)
VALUES ('Confirmada');

INSERT INTO ESTADOS (Nombre_Estado)
VALUES ('Cancelada');

INSERT INTO ESTADOS (Nombre_Estado)
VALUES ('Finalizada');
```

Salida de Script | Resultado de la Consulta | Resultado de la Consulta 1

Todas las Filas Recuperadas: 43 en 0,178 segundos

ID_USUARIO	NOMBRE	APELLIDO_1	APELLIDO_2	CORREO	TELEFONO	CONTRASENA_HASH	ID_ROL
1	Flor	Martinez	Garcia	flor@correo.com	8888-1111	hash123	1
2	Ana	Lopez	Sanchez	ana@correo.com	8888-2222	hash456	2
3	Carlos	Perez	Mora	carlos@correo.com	8888-3333	hash789	3
4	Alhana	Diaz	Aguirre	alhana@correo.com	8888-4444	hash132	2
5	Stephanie	Brenes	Araya	brenes@correo.com	8888-5555	hash856	2
6	Carlos	Gutierrez	Quiros	gutierrez@correo.com	8888-6666	hash098	2
7	Sandra	Rojas	Brenes	sandra@correo.com	8877-1234	hash139	2
8	Stephanie	Brenes	Dobles	dobles@correo.com	8877-2345	hash176	2
9	Jose	Hernandez	Perez	hernandez@correo.com	8877-3456	hash086	2
10	Martha	Fonseca	Artavia	martha@correo.com	8877-4567	hash443	2
11	Manuel	Cruz	Cordoba	cruz@correo.com	8877-5678	hash406	2
12	Alejandro	Leiva	Quiros	quiros@correo.com	8877-5679	hash996	2
13	Christian	Marin	Hernandez	christian@correo.com	8877-6789	hash566	2
14	Roberto	Brenes	Araya	roberto@correo.com	8877-7890	hash444	2

Resultado de la Consulta

Todas las Filas Recuperadas: 2 en 0,193 segundos

ID_RESERVA	ID_USUARIO	ID_SALA	ID_ESTADO	FECHA	HORA_INICIO	HORA_FIN	COSTO_TOTAL
1	1	2	1	2 15/03/26	15/03/26 08:00:00,000000000 AM	15/03/26 10:00:00,000000000 AM	30000
2	2	2	2	1 16/03/26	16/03/26 01:00:00,000000000 PM	16/03/26 03:00:00,000000000 PM	20000

Figura 4. *Inserción de datos en las tablas del sistema.*

El poblamiento permitió verificar que la base de datos responde correctamente a las restricciones establecidas. Por ejemplo, no es posible registrar una reserva si el usuario o la

sala no existen previamente, lo cual demuestra el correcto funcionamiento de las llaves foráneas.

Asimismo, se comprobó que los pagos únicamente pueden asociarse a reservas válidas, garantizando la coherencia de la información financiera dentro del sistema.

Este proceso es fundamental, ya que permite validar el comportamiento del sistema antes de su implementación final, asegurando que la estructura de datos sea funcional y consistente (Microsoft, 2023).

4.6 Validación del sistema mediante consultas

Con el objetivo de validar el correcto funcionamiento de la base de datos, se realizaron diversas consultas SQL utilizando la instrucción SELECT.

Estas consultas permitieron visualizar los datos almacenados en cada tabla y comprobar que las relaciones entre ellas se encuentran correctamente definidas. Además, se realizaron consultas con múltiples JOIN, lo cual permitió integrar información de distintas entidades y obtener resultados más completos.

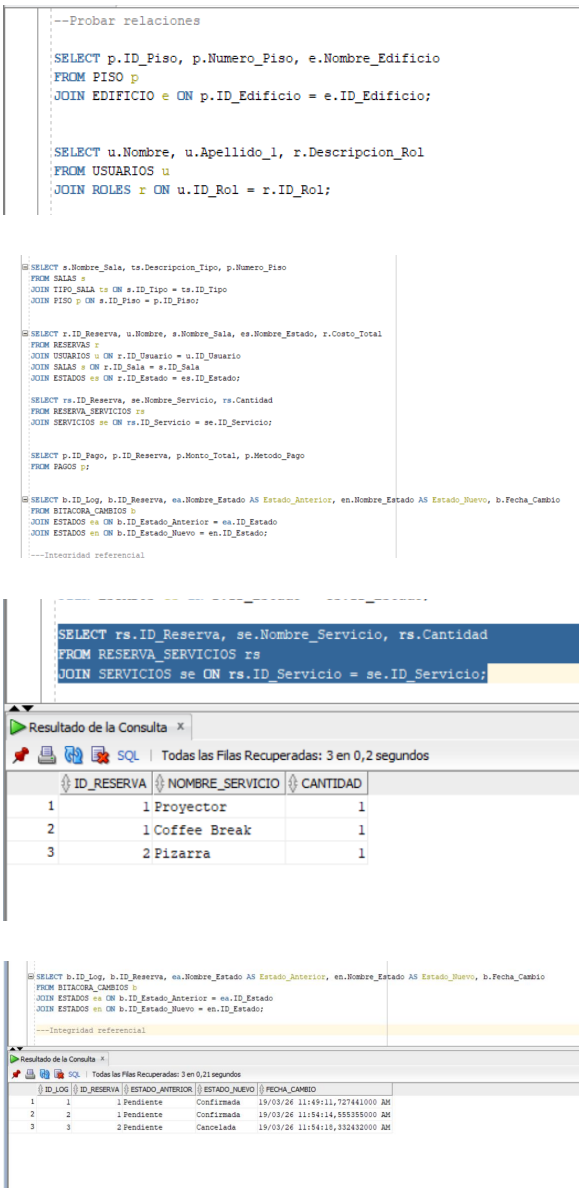


***Figura 5**. Consulta de validación con múltiples tablas relacionadas.*

Por ejemplo, se realizaron consultas que permiten visualizar el detalle completo de una reserva, incluyendo el nombre del usuario, la sala seleccionada, el estado de la reserva y el monto asociado. Este tipo de consultas demuestra la correcta implementación del modelo relacional.

Adicionalmente, se llevaron a cabo pruebas de integridad, como la inserción de datos inválidos o la eliminación de registros relacionados. En estos casos, el sistema generó errores controlados, lo cual confirma que las restricciones definidas funcionan adecuadamente.

El uso de consultas SQL es una práctica esencial para validar la calidad de los datos y asegurar el correcto funcionamiento del sistema (W3Schools, 2024).

4.7 Interfaces del sistema (Implementado en Oracle APEX)

En esta seccion del proyecto que representan las principales interfaces del sistema. Estas ya son capturas reales de la aplicacion en funcionamiento.

A continuación, se describen detalladamente las interfaces desarrolladas:

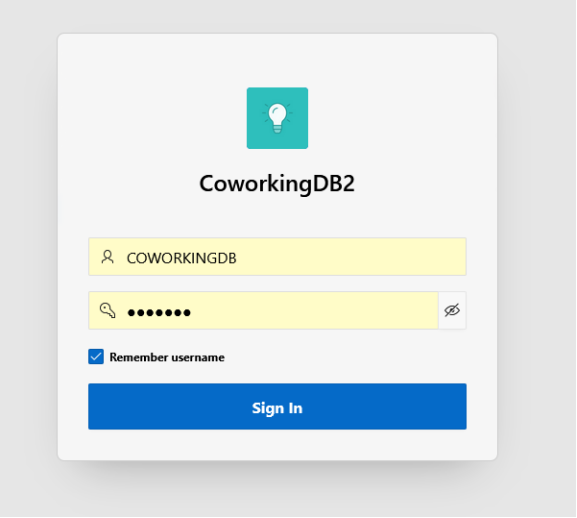


***Figura 6**. Pantalla de inicio de sesión*

Esta interfaz permite al usuario ingresar al sistema mediante el uso de credenciales, específicamente correo electrónico y contraseña. Se encuentra directamente relacionada con la tabla USUARIOS, ya que valida la información almacenada en la base de datos.

El diseño incluye campos de entrada, botones de acceso y una estructura simple que facilita la interacción del usuario. Su función principal es garantizar la autenticación y seguridad del sistema.

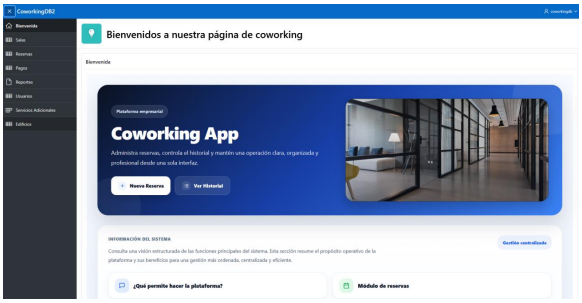


***Figura 7.** Panel principal del sistema (Dashboard)*

El dashboard representa la pantalla principal del sistema una vez que el usuario ha iniciado sesión. En esta interfaz se presenta un resumen general del sistema, incluyendo accesos rápidos a funcionalidades como reservas, salas y pagos.

Además, permite visualizar información relevante de manera rápida, facilitando la navegación y mejorando la experiencia del usuario.

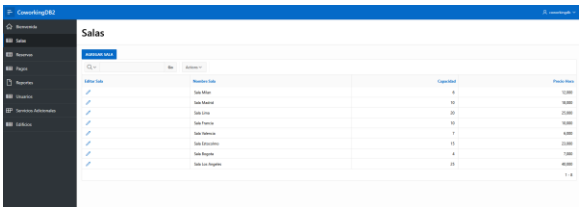


Figura 8. *Vista de salas disponibles*

Esta interfaz muestra las salas disponibles dentro del coworking, incluyendo información como nombre, capacidad, tipo de sala y precio por hora.

Se encuentra directamente relacionada con las tablas SALAS y TIPO_SALA. Su objetivo es permitir al usuario analizar las opciones disponibles antes de realizar una reserva.

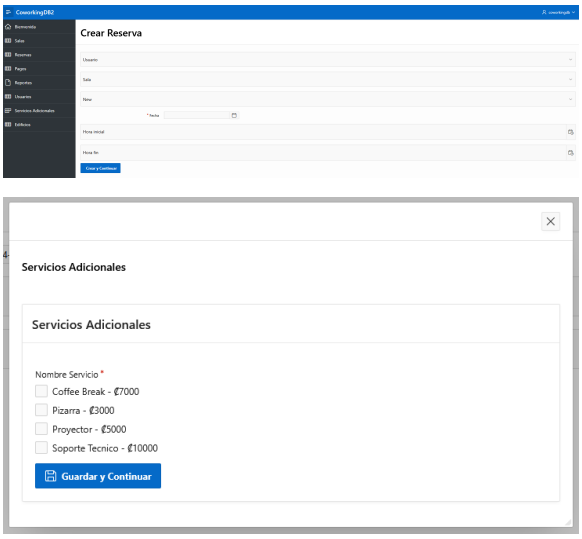


Figura 9. *Formulario de creación de reservas*

Esta interfaz permite al usuario registrar una nueva reserva ingresando datos como la sala seleccionada, la fecha y el horario.

Se vincula directamente con la tabla RESERVAS, así como con las tablas relacionadas (USUARIOS, SALAS y ESTADOS). Esta funcionalidad es una de las más importantes del sistema, ya que permite gestionar el uso de las salas.

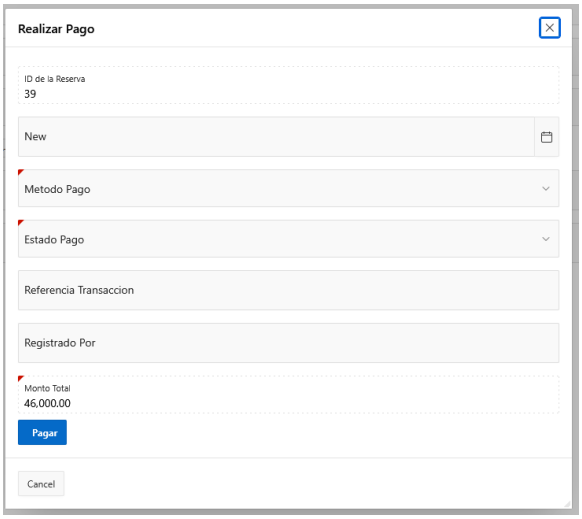


Figura 10. *Pantalla de pagos*

Esta interfaz permite registrar el pago asociado a una reserva. Incluye información como monto total, método de pago y fecha.

Se encuentra relacionada con la tabla PAGOS, permitiendo almacenar y gestionar la información financiera del sistema.

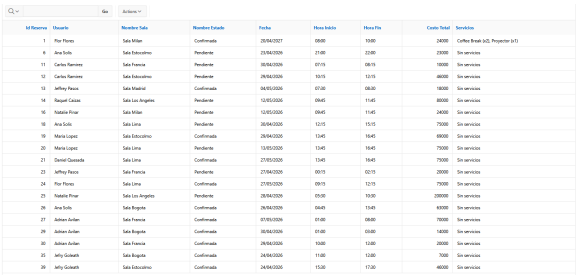


Figura 11. *Historial de reservas*

Esta interfaz permite visualizar el historial de reservas realizadas por el usuario, incluyendo detalles como fechas, salas, estados y costos.

Está relacionada con múltiples tablas, principalmente RESERVAS, SALAS y ESTADOS, lo que permite ofrecer una vista completa de la actividad del usuario dentro del sistema.

4.8 Flujo general del sistema

El sistema sigue un flujo lógico que inicia con la autenticación del usuario. Una vez dentro del sistema, el usuario puede consultar las salas disponibles, realizar reservas, agregar servicios adicionales y efectuar pagos.

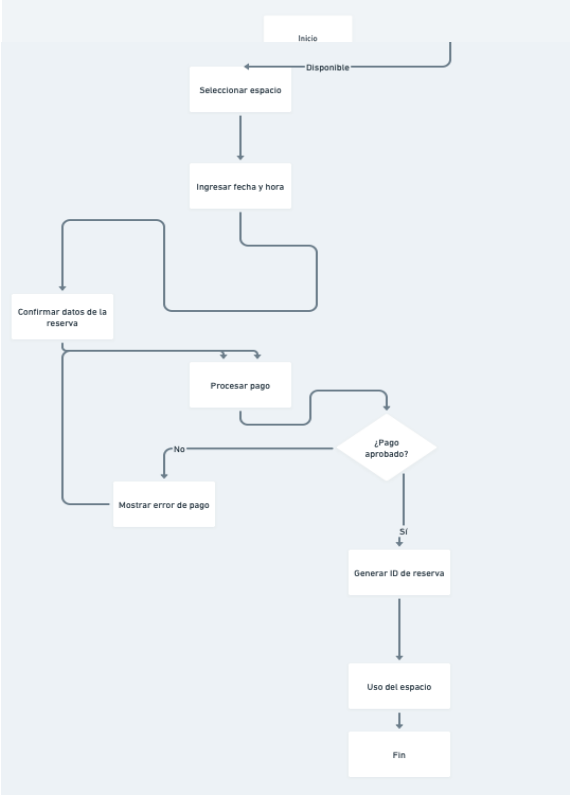


Figura 12. *Flujo del sistema*

Cada una de estas acciones se encuentra respaldada por la base de datos. Por ejemplo, la creación de una reserva genera un registro en la tabla RESERVAS, mientras que los servicios adicionales se almacenan en RESERVA_SERVICIOS y los pagos en la tabla PAGOS.

Este flujo demuestra la integración entre las distintas entidades del sistema y resalta la importancia de una estructura de datos bien definida. Una correcta organización permite que el sistema sea escalable, eficiente y fácil de mantener (IBM, 2023).

5. ANÁLISIS DE RESULTADOS

A continuación, se detallarán los procedimientos utilizados para el desarrollo del proyecto, esto incluye procedimientos con cursores, triggers y consultas con expresiones regulares.

5.1. Estructura de procedimientos utilizados

5.1.1 Paquetes y CRUD.

1. PKG_CATALOGOS: Gestiona el CRUD de las tablas maestras como Edificios, Pisos, Roles, Estados y Servicios.

- sp_insertar_edificio: Recibe como s el nombre del edificio y su dirección.

El procedimiento valida que el nombre no esté duplicado.

- sp_actualizar_edificio: Ocupa el ID_EDIFICIO para ubicar el registro y los nuevos valores de nombre o dirección.
- sp_eliminar_edificio:Recibe el ID_EDIFICIO para borrar el registro.

2.PKG_RESERVAS: Se enfoca en la creación y gestión de reservas incluyendo la lógica para insertar registros y agarrar el ID generado para los procesos.

3.PKG_USUARIOS: Administra la creación, actualización y eliminación de usuarios dando de manera segura que se cumplan las validaciones de perfil.

4. PKG_PAGOS: Controla el registro de transacciones financieras y la actualización de estados de pago.

5.1.2 Consultas con expresiones regulares.

Se agregaron cuatro consultas utilizando expresiones regulares mediante la función REGEXP_LIKE.

Las consultas permiten buscar información siguiendo patrones específicos dentro de los datos almacenados en la base de datos.

1.Validación de correos eléctronicos:

Se implementó una expresión regular para validar que los correos electrónicos sigan una estructura válida evitando errores de sintaxis comunes en el registro de usuarios.

- Expresión:'^[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+(\.[A-Za-z0-9-]+)+$'

2. Validación de Métodos de Pago: Para evitar entradas de texto inconsistentes en la tabla de cobros se definió un patrón que restringe el campo solo para los métodos autorizados por el admin.

- Expresión:'^(Tarjeta|SINPE|Transferencia|Efectivo)$'

3. Estructura de Telefonía Celular (TELEFONO):
Se configuró una regla para estandarizar el registro de los numeros de telefono de los clientes permitiendo únicamente el formato numérico del pais sin espacios.

- Expresión: '^[0-9]{8}$'

4.Política de contraseñas (CONTRASENA_HASH):
Se se aplicaron varias expresiones regulares para validar que las contraseñas de los usuarios no sean vulnerables.

Expresiones:

- '[A-Z]': Obliga a incluir al menos una letra mayúscula.
- '[a-z]': Obliga a incluir al menos una letra minúscula.
- '[0-9]': Obliga a incluir al menos un valor numérico.
- '[^A-Za-z0-9]': Obliga a incluir al menos un símbolo o carácter especial

5.1.3 Triggers

1. trg_reservas_validar_disponibilidad: Su función es evitar que una sala se reserve dos veces en el mismo horario. Cada vez que alguien intenta insertar una nueva reserva en la tabla de reservar se ejecuta el trigger se ejecuta antes de guardar el registro y le revisa si ya existe otra reserva para la misma sala y fecha y también verifica si las horas se cruzan con la nueva reserva. Si encuentra una reserva pendiente o confirmada en ese mismo horario, el sistema genera un error y no permite guardar la nueva reserva.

2. trg_usuarios_bi y trg_usuarios_bu: Estos triggers se ejecutan en los eventos BEFORE INSERT y BEFORE UPDATE de la tabla USUARIOS funcionando como una linea de protección para limpieza en la base de datos. Se usa la función TRIM para eliminar espacios accidentales y LOWER para estandarizar los correos electrónicos. Tambien integran llamadas a expresiones regulares para confirmar que las contraseñas cumplan con el estándar (letras, números y símbolos)

3. trg_reservas_calcular_costo: El trigger agarra el registro y por una consulta a la tabla SALAS obtiene el precio_hora vigente despues hace un cálculo donde resta la hora_inicio de la hora_fin para tener la duración total y al final hace una multiplicación de las horas reservadas por el precio por hora de la sala y guarda el resultado en el campo costo_total.

4.TRIGGER TRG_AUDITORIA_ESTADOS: Cada vez que un administrador o el sistema cambia el estado de una reserva (de 'Pendiente' a 'Cancelada' o 'Pagada') este trigger agarra de forma automatica el ID de la reserva el estado antiguo el nuevo estado y la fecha/hora del movimiento insertándolos en la tabla BITACORA_CAMBIOS.

5.1.5 Cursores

1.sp_listar_usuarios_cursor: Usa un cursor interno (c_usuarios) que recorre la tabla USUARIOS ordenándolos por ID permitiendo dar un formato de salida entendible para el usuario.

2.sp_lista_reservas_cursor: El cursor (c_reservas) realiza un JOIN entre las tablas

RESERVAS, USUARIOS y SALAS. Esto permite mostrar en una sola línea el nombre del cliente, la sala específica y el costo total calculado por el sistema.

3.sp_reporte_reservas_por_fecha: Usa un cursor (c_reporte) que usa la función TRUNC para comparar fechas recuperando solo los registros que coinciden con el día solicitado por el usuario.

4. sp_reporte_salas_mas_reservadas: Usa un cursor (c_salas) que usa funciones de (COUNT) y (GROUP BY) pasa por el resultado de un LEFT JOIN para contar cuántas veces ha sido utilizada cada sala ordenando los resultados de mayor a menor demanda.

5.1.6 Diccionario de Datos

En esta sección se detalla la estructura de las tablas principales del sistema:

Tabla	Columna	Tipo de Dato	Descripción
EDIFICIO	id_edificio	NUMBER (PK)	Identificador único de la sede.
	nombre_edificio	VARCHAR2(100)	Nombre de la ubicación.
	direccion	VARCHAR2(255)	Dirección física completa.
PISO	id_piso	NUMBER (PK)	Identificador del nivel del edificio.
	numero_piso	NUMBER(2)	Número de planta (0-99).
	id_edificio	NUMBER (FK)	Relación con la tabla EDIFICIO.
ROLES	id_rol	NUMBER (PK)	ID del perfil de usuario.
	descripcion_rol	VARCHAR2(50)	Nombre del rol (Admin, Cliente, etc.).
USUARIOS	id_usuario	NUMBER (PK)	Identificador único del usuario.
	nombre	VARCHAR2(50)	Nombre(s) del usuario.
	apellido_1	VARCHAR2(50)	Primer apellido.
	correo	VARCHAR2(100)	Email único (Validado con REGEXP)
	telefono	VARCHAR2(20)	Número telefónico (8 dígitos)
	id_rol	NUMBER (FK)	Perfil asignado al usuario.
TIPO_SALA	id_tipo	NUMBER (PK)	ID de categoría de sala.
	descripcion_tipo	VARCHAR2(50)	Ej: Sala de Reunión, Privada.
SALAS	id_sala	NUMBER (PK)	Identificador de la sala física.
	nombre_sala	VARCHAR2(100)	Nombre único de la sala.
	capacidad	NUMBER(3)	Aforo máximo permitido.
	precio_hora	NUMBER(8,2)	Costo base por hora de uso.
	id_piso	NUMBER (FK)	Ubicación física (piso) de la sala.
ESTADOS	id_estado	NUMBER (PK)	ID del estado de la reserva.
	nombre_estado	VARCHAR2(50)	Pendiente, Confirmada, Cancelada.
RESERVAS	id_reserva	NUMBER (PK)	Identificador de la transacción.
	fecha	DATE	Fecha programada para el evento.
	hora_inicio	TIMESTAMP	Tiempo exacto de inicio.
	hora_fin	TIMESTAMP	Tiempo exacto de finalización.
	costo_total	NUMBER(10,2)	Monto calculado (Trigger trg_reservas_calcular_costo)
SERVICIOS	id_servicio	NUMBER (PK)	ID del servicio adicional.
	nombre_servicio	VARCHAR2(100)	Ej: Proyector, Catering.
	precio_adicional	NUMBER(8,2)	Costo por unidad del servicio.
RESERVA_SERVICIOS	id_reserva	NUMBER (PK, FK)	Asociación con la reserva.
	id_servicio	NUMBER (PK, FK)	Asociación con el servicio.
	cantidad	NUMBER(2)	Cantidad solicitada
PAGOS	id_pago	NUMBER (PK)	ID del comprobante de pago.
	monto_total	NUMBER(10,2)	Total (Reserva + Servicios).
	metodo_pago	VARCHAR2(50)	Tarjeta, SINPE, etc. (Validado REGEXP)
BITACORA_CAMBIOS	id_log	NUMBER (PK)	ID del evento de auditoría.
	id_reserva	NUMBER (FK)	Reserva que sufrió el cambio.
	id_estado_nuevo	NUMBER (FK)	Nuevo estado asignado.

5.2 Repositorio

A continuación, se adjunta el link del repositorio público utilizado para el desarrollo de este proyecto: https://github.com/J-Goleath/SC_504_Proyecto_G1.git

6. CONCLUSIONES

Durante el desarrollo de este proyecto se logró proponer el diseño de un sistema de gestión de salas para espacios de coworking, basado en el uso de una base de datos relacional. Durante el proceso se identificaron las necesidades principales relacionadas con la organización de reservas y el control del uso de las salas, lo cual permitió estructurar adecuadamente la información necesaria dentro del sistema.

Gracias a la implementación del modelo relacional en la base de datos permitió comprobar que las relaciones entre las tablas funcionan correctamente mediante el uso de llaves primarias, llaves foráneas y restricciones de integridad. Se realizaron pruebas para poder verificar y comprobar que el sistema puede gestionar reservas de forma ordenada y evitar inconsistencias en la información.

Por otra parte, el desarrollo de mockups y la definición del flujo general del sistema permitieron visualizar cómo interactuarán los usuarios con la plataforma, lo cual ayuda a entender mejor el funcionamiento del sistema antes de su implementación completa.

En general, el proyecto permitió aplicar los conocimientos adquiridos en el curso de Lenguaje de Bases de Datos en un contexto práctico. Además, demuestra cómo el uso de bases de datos puede contribuir a mejorar la organización y administración de recursos dentro de espacios de trabajo compartidos como el coworking.

7. RECOMENDACIONES

Como posibles mejoras del sistema, se recomienda continuar fortaleciendo el diseño de la base de datos, por ejemplo, incorporando más restricciones y validaciones que permitan evitar conflictos en los horarios de reservas y asegurar una mayor consistencia de los datos.

También sería conveniente implementar procedimientos almacenados, funciones o triggers que automaticen ciertos procesos dentro de la base de datos, como validaciones de reservas o registro de cambios.

Finalmente, se podría ampliar el modelo de datos para incluir nuevas funcionalidades, como la gestión de múltiples sedes o un control más detallado de los servicios del coworking.

5 Referencias

[1] Amazon Web Services. (2023). *Entity-relationship modeling*. https://aws.amazon.com/what-is/entity-relationship-diagram/

[2] IBM. (2023). *What is requirements gathering?* https://www.ibm.com/topics/requirements-gathering

[3] Lucidchart. (2023). *What is an ERD?* https://www.lucidchart.com/pages/er-diagrams

[4] MySQL. (2024). *MySQL documentation*. https://dev.mysql.com/doc/

[5] Oracle. (2023). *Database schema design guide*. https://docs.oracle.com/en/database/

[6] PostgreSQL Global Development Group. (2023). *Best practices for database design*. https://www.postgresql.org/docs/

[7] Tecnológico de Costa Rica. (2023). *Tipos de investigación*. https://www.tec.ac.cr

[8] Universidad de Valencia. (2022). *Investigación descriptiva*. https://www.uv.es

[9] Canva. (2024). *Diagramas en línea*. https://www.canva.com/es_mx/diagramas/

[10] Google. (2024). *Cómo buscar mejor en internet*. https://support.google.com/websearch/answer/4540094

[11] Notion. (2023). *Centro de ayuda*. https://www.notion.so/help

[12] Lucidchart. (2024). *ER diagram examples*. https://www.lucidchart.com/pages/examples/er-diagram-examples

[13] Microsoft Learn. (2024). *Database design basics*. https://learn.microsoft.com/en-us/office/troubleshoot/access/database-design-basics

[14] Nexudus. (2024). *Coworking management platform*. https://www.nexudus.com

[15] Coronel, C., & Morris, S. (2019). Database systems: Design, implementation, and management (13th ed.). Cengage Learning.

[16] Laudon, K. C., & Laudon, J. P. (2021). Management information systems: Managing the digital firm (16th ed.). Pearson Education.

[17] Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill Education.

[18] IBM. (2023). *What is a database?*

https://www.ibm.com/topics/databases

[19] Interaction Design Foundation. (2024). *What are mockups?* https://www.interaction-design.org/literature/topics/mockups

[20] Microsoft. (2023). *Database design basics*. https://learn.microsoft.com/en-us/office/troubleshoot/access/database-design-basics

[21] Oracle. (2023). *Database Concepts*. https://docs.oracle.com/en/database/oracle/oracle-database/

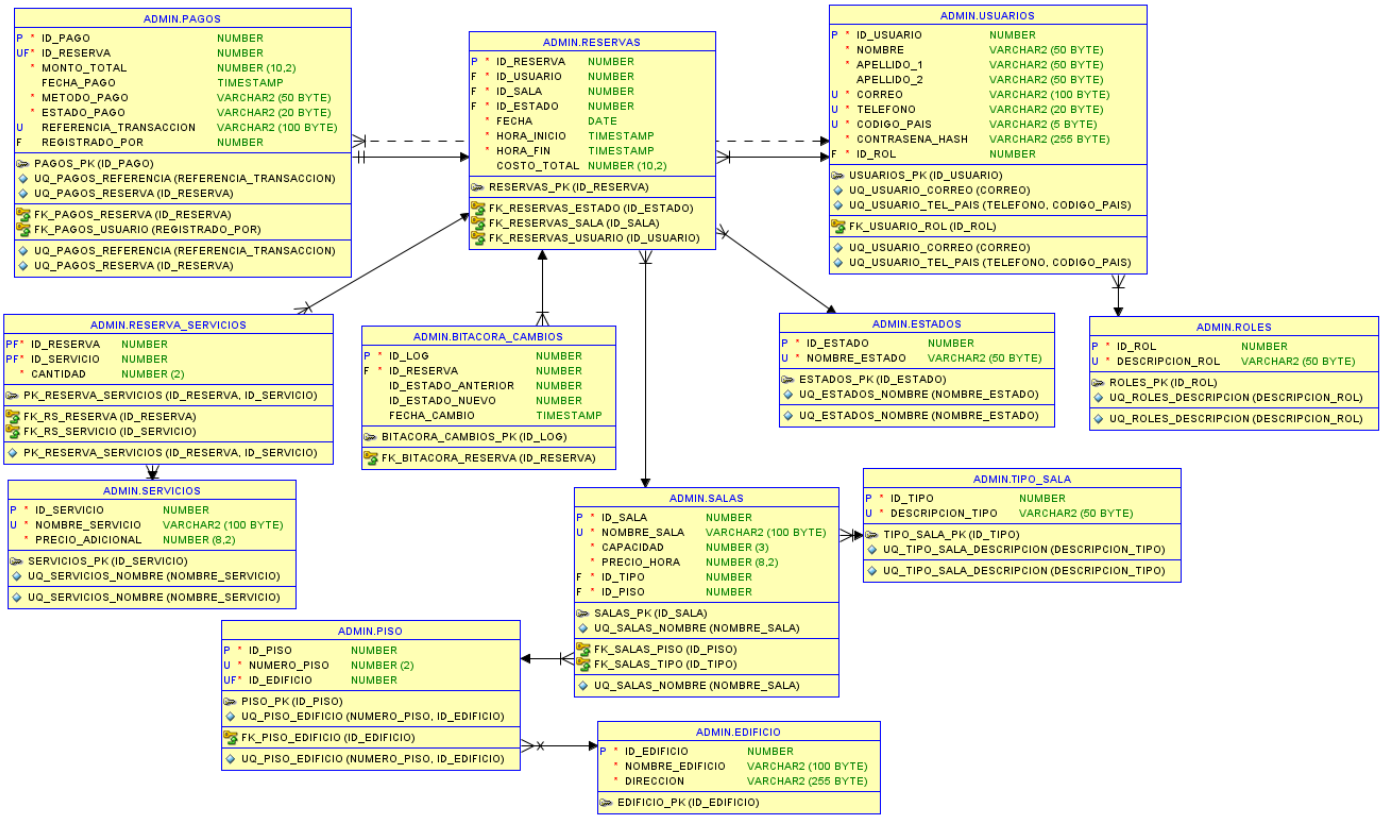
[22] W3Schools. (2024). *SQL Tutorial*. https://www.w3schools.com/sql/

[23] Figma. (2024). *Figma: The collaborative interface design tool*. https://www.figma.com

[24] rwestMSFT. (s. f.). Cursores (SQL server) - SQL server. Microsoft Learn. https://learn.microsoft.com/es-es/sql/relational-databases/cursors?view=sql-server-ver17

6 Anexos

Anexo 1: Diagrama ER



Anexo 2: Creacion de tablas

Los anexos contienen evidencia del desarrollo e implementación del sistema, incluyendo scripts SQL, datos de prueba, consultas y resultados de ejecución.

Hoja de Trabajo | Generador de Consultas

```
CREATE TABLE PISO (
    ID_Piso     NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Numero_Piso NUMBER NOT NULL,
    ID_Edificio NUMBER NOT NULL,
    CONSTRAINT fk_piso_edificio FOREIGN KEY (ID_Edificio)
        REFERENCES EDIFICIO(ID_Edificio) ON DELETE CASCADE
);

-- 3. ROLES
CREATE TABLE ROLES (
    ID_Rol          NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Descripcion_Rol VARCHAR2(50) NOT NULL
);

-- 4. USUARIOS
-- 4. USUARIOS
-- 3. ROLES
CREATE TABLE ROLES (
    ID_Rol          NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Descripcion_Rol VARCHAR2(50) NOT NULL
);
```

Salida de Script | Tarea terminada en 0,425 segundos

```
Table EDIFICIO creado.


Table PISO creado.


Table ROLES creado.
```

Anexo 3 Creacion de tablas

```sql
CREATE TABLE USUARIOS (
    ID_Usuario      NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Nombre          VARCHAR2(100) NOT NULL,
    Apellido_1      VARCHAR2(100) NOT NULL,
    Apellido_2      VARCHAR2(100) NOT NULL,
    Correo          VARCHAR2(100) UNIQUE NOT NULL,
    Telefono        VARCHAR2(20),
    Contrasena_Hash VARCHAR2(255) NOT NULL,
    ID_Rol          NUMBER,
    CONSTRAINT fk_usuario_rol
        FOREIGN KEY (ID_Rol) REFERENCES ROLES(ID_Rol) ON DELETE SET NULL
);

-- 5. TIPO_SALA
CREATE TABLE TIPO_SALA (
    ID_Tipo          NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Descripcion_Tipo VARCHAR2(50) NOT NULL
);

-- 6. SALAS
CREATE TABLE SALAS (
    ID_Sala     NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Nombre_Sala VARCHAR2(100) NOT NULL,
    Capacidad   NUMBER NOT NULL,
    Precio_Hora NUMBER(10,2) NOT NULL,
    ID_Tipo     NUMBER,
    ID_Piso     NUMBER,
    CONSTRAINT fk_sala_tipo FOREIGN KEY (ID_Tipo)
        REFERENCES TIPO_SALA(ID_Tipo) ON DELETE SET NULL,
    CONSTRAINT fk_sala_piso FOREIGN KEY (ID_Piso)
        REFERENCES PISO(ID_Piso) ON DELETE SET NULL
);
```

Anexo 4 Creacion de tablas

```sql
-- 7. ESTADOS
CREATE TABLE ESTADOS (
    ID_Estado     NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Nombre_Estado VARCHAR2(50) NOT NULL
);

-- 8. RESERVAS
CREATE TABLE RESERVAS (
    ID_Reserva      NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    ID_Usuario      NUMBER NOT NULL,
    ID_Sala         NUMBER NOT NULL,
    ID_Estado       NUMBER NOT NULL,
    Fecha           DATE NOT NULL,
    Hora_Inicio     TIMESTAMP NOT NULL,
    Hora_Fin        TIMESTAMP NOT NULL,
    Costo_Total     NUMBER(12,2),
    CONSTRAINT fk_reserva_usuario FOREIGN KEY (ID_Usuario) REFERENCES USUARIOS(ID_Usuario),
    CONSTRAINT fk_reserva_sala FOREIGN KEY (ID_Sala) REFERENCES SALAS(ID_Sala),
    CONSTRAINT fk_reserva_estado FOREIGN KEY (ID_Estado) REFERENCES ESTADOS(ID_Estado)
);

-- 9. SERVICIOS
CREATE TABLE SERVICIOS (
    ID_Servicio     NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    Nombre_Servicio VARCHAR2(100) NOT NULL,
    Precio_Adicional NUMBER(10,2) NOT NULL
);

-- 10. RESERVA_SERVICIOS
CREATE TABLE RESERVA_SERVICIOS (
    ID_Reserva  NUMBER NOT NULL,
    ID_Servicio NUMBER NOT NULL,
    Cantidad    NUMBER DEFAULT 1,
    PRIMARY KEY (ID_Reserva, ID_Servicio),
    CONSTRAINT fk_rs_reserva FOREIGN KEY (ID_Reserva) REFERENCES RESERVAS(ID_Reserva) ON DELETE CASCADE,
```

Anexo 5 Creacion de tablas

```
    CONSTRAINT fk_rs_reserva FOREIGN KEY (ID_Reserva) REFERENCES RESERVAS(ID_Reserva) ON DELETE CASCADE,
    CONSTRAINT fk_rs_servicio FOREIGN KEY (ID_Servicio) REFERENCES SERVICIOS(ID_Servicio)
);

-- 11. PAGOS
CREATE TABLE PAGOS (
    ID_Pago       NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    ID_Reserva    NUMBER NOT NULL,
    Monto_Total   NUMBER(10,2) NOT NULL,
    Fecha_Pago    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    Metodo_Pago   VARCHAR2(50),
    CONSTRAINT fk_pago_reserva FOREIGN KEY (ID_Reserva) REFERENCES RESERVAS(ID_Reserva)
);

-- 12. BITACORA_CAMBIOS
CREATE TABLE BITACORA_CAMBIOS (
    ID_Log              NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    ID_Reserva          NUMBER,
    ID_Estado_Anterior  NUMBER,
    ID_Estado_Nuevo     NUMBER,
    Fecha_Cambio        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    CONSTRAINT fk_bitacora_reserva
        FOREIGN KEY (ID_Reserva) REFERENCES RESERVAS(ID_Reserva) ON DELETE CASCADE,

    CONSTRAINT fk_bitacora_estado_ant
        FOREIGN KEY (ID_Estado_Anterior) REFERENCES ESTADOS(ID_Estado),

    CONSTRAINT fk_bitacora_estado_nue
        FOREIGN KEY (ID_Estado_Nuevo) REFERENCES ESTADOS(ID_Estado)
);
```

Anexo 6 Confirmacion de funcionamiento de script

```
Salida de Script
Tarea terminada en 0,192 segundos

Table SALAS creado.

Table ESTADOS creado.

Table RESERVAS creado.

Table SERVICIOS creado.

Table RESERVA_SERVICIOS creado.

Table PAGOS creado.

Table BITACORA_CAMBIOS creado.
```

Anexo 7 Insercion de datos

```sql
-- EDIFICIO
INSERT INTO EDIFICIO (Nombre_Edificio, Direccion)
VALUES ('Coworking Central', 'San Jose, Costa Rica');

INSERT INTO EDIFICIO (Nombre_Edificio, Direccion)
VALUES ('Coworking Escazu', 'Escazu, San Jose');

-- PISO
INSERT INTO PISO (Numero_Piso, ID_Edificio)
VALUES (1, 1);

INSERT INTO PISO (Numero_Piso, ID_Edificio)
VALUES (2, 1);

INSERT INTO PISO (Numero_Piso, ID_Edificio)
VALUES (1, 2);

-- ROLES
INSERT INTO ROLES (Descripcion_Rol)
VALUES ('Administrador');

INSERT INTO ROLES (Descripcion_Rol)
VALUES ('Cliente');

INSERT INTO ROLES (Descripcion_Rol)
```

Anexo 8 Insercion de datos

Hoja de Trabajo | Generador de Consultas

```sql
-- USUARIOS
INSERT INTO USUARIOS (Nombre, Apellido_1, Apellido_2, Correo, Telefono, Contrasena_Hash, ID_Rol)
VALUES ('Flor', 'Martinez', 'Garcia', 'flor@correo.com', '8888-1111', 'hash123', 1);

INSERT INTO USUARIOS (Nombre, Apellido_1, Apellido_2, Correo, Telefono, Contrasena_Hash, ID_Rol)
VALUES ('Ana', 'Lopez', 'Sanchez', 'ana@correo.com', '8888-2222', 'hash456', 2);

INSERT INTO USUARIOS (Nombre, Apellido_1, Apellido_2, Correo, Telefono, Contrasena_Hash, ID_Rol)
VALUES ('Carlos', 'Perez', 'Mora', 'carlos@correo.com', '8888-3333', 'hash789', 3);

-- USUARIOS EXTRA (CORREGIDOS)
SELECT USER FROM dual;
INSERT INTO USUARIOS VALUES (DEFAULT, 'Alhana', 'Diaz', 'Aguirre', 'alhana@correo.com', '8888-4444', 'hash132', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Stephanie', 'Brenes', 'Araya', 'brenes@correo.com', '8888-5555', 'hash856', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Carlos', 'Gutierrez', 'Quiros', 'gutierrez@correo.com', '8888-6666', 'hash098', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Sandra', 'Rojas', 'Brenes', 'sandra@correo.com', '8877-1234', 'hash139', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Stephanie', 'Brenes', 'Dobles', 'dobles@correo.com', '8877-2345', 'hash176', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Jose', 'Hernandez', 'Perez', 'hernandez@correo.com', '8877-3456', 'hash086', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Martha', 'Fonseca', 'Artavia', 'martha@correo.com', '8877-4567', 'hash443', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Manuel', 'Cruz', 'Cordoba', 'cruz@correo.com', '8877-5678', 'hash406', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Alejandro', 'Leiva', 'Quiros', 'quiros@correo.com', '8877-5679', 'hash998', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Christian', 'Marin', 'Hernandez', 'christian@correo.com', '8877-6789', 'hash566', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Roberto', 'Brenes', 'Araya', 'roberto@correo.com', '8877-7890', 'hash444', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Marco', 'Perez', 'Ramirez', 'marco@correo.com', '8788-1122', 'hash009', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Emilce', 'Romero', 'Martinez', 'emilce@correo.com', '8788-2233', 'hash223', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'David', 'Araya', 'Figueres', 'david@correo.com', '8788-3344', 'hash445', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Marvin', 'Fernandez', 'Robles', 'marvin@correo.com', '8788-4455', 'hash454', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Kevin', 'Abarca', 'Coto', 'coto@correo.com', '8788-5566', 'hash109', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Naomy', 'Alvarado', 'Calderon', 'naomy@correo.com', '8788-6677', 'hash450', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Jimena', 'Mata', 'Barrantes', 'jimena@correo.com', '8788-7788', 'hash012', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Tamara', 'Ramos', 'Morales', 'tamara1@correo.com', '8788-8899', 'hash165', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Myriam', 'Aguirre', 'Chavarria', 'tamara2@correo.com', '8765-8899', 'hash165', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Catalina', 'Navarro', 'Marin', 'tamara3@correo.com', '8766-8899', 'hash887', 2);
INSERT INTO USUARIOS VALUES (DEFAULT, 'Patricia', 'Valencia', 'Mendoza', 'patri@correo.com', '8767-8899', 'hash889', 2);
```

Anexo 9 Insercion de datos

```sql
-- RESERVA_SERVICIOS
INSERT INTO RESERVA_SERVICIOS (ID_Reserva, ID_Servicio, Cantidad)
VALUES (1, 1, 1);

INSERT INTO RESERVA_SERVICIOS (ID_Reserva, ID_Servicio, Cantidad)
VALUES (1, 2, 1);

INSERT INTO RESERVA_SERVICIOS (ID_Reserva, ID_Servicio, Cantidad)
VALUES (2, 3, 1);

-- PAGOS
INSERT INTO PAGOS (ID_Reserva, Monto_Total, Metodo_Pago)
VALUES (1, 43000, 'Tarjeta');

INSERT INTO PAGOS (ID_Reserva, Monto_Total, Metodo_Pago)
VALUES (2, 23000, 'SINPE');

-- BITACORA_CAMBIOS
INSERT INTO BITACORA_CAMBIOS (ID_Reserva, ID_Estado_Anterior, ID_Estado_Nuevo)
VALUES (1, 1, 2);

INSERT INTO BITACORA_CAMBIOS (ID_Reserva, ID_Estado_Anterior, ID_Estado_Nuevo)
VALUES (2, 1, 3);

COMMIT;
```

Anexo 10 Insercion de datos

```sql
-- TIPO_SALA
INSERT INTO TIPO_SALA (Descripcion_Tipo)
VALUES ('Sala de reuniones');

INSERT INTO TIPO_SALA (Descripcion_Tipo)
VALUES ('Oficina privada');

INSERT INTO TIPO_SALA (Descripcion_Tipo)
VALUES ('Sala de capacitacion');

-- SALAS
INSERT INTO SALAS (Nombre_Sala, Capacidad, Precio_Hora, ID_Tipo, ID_Piso)
VALUES ('Sala A', 10, 15000, 1, 1);

INSERT INTO SALAS (Nombre_Sala, Capacidad, Precio_Hora, ID_Tipo, ID_Piso)
VALUES ('Sala B', 4, 10000, 2, 2);

INSERT INTO SALAS (Nombre_Sala, Capacidad, Precio_Hora, ID_Tipo, ID_Piso)
VALUES ('Sala C', 20, 25000, 3, 3);

-- ESTADOS
INSERT INTO ESTADOS (Nombre_Estado)
VALUES ('Pendiente');

INSERT INTO ESTADOS (Nombre_Estado)
VALUES ('Confirmada');

INSERT INTO ESTADOS (Nombre_Estado)
VALUES ('Cancelada');

INSERT INTO ESTADOS (Nombre_Estado)
VALUES ('Finalizada');
```

Anexo 11 Confirmacion de insercion de datos

Salida de Script | Resultado de la Consulta | Resultado de la Consulta 1

SQL | Todas las Filas Recuperadas: 43 en 0,178 segundos

	ID_USUARIO	NOMBRE	APELLIDO_1	APELLIDO_2	CORREO	TELEFONO	CONTRASENA_HASH	ID_ROL
1	1	Flor	Martinez	Garcia	flor@correo.com	8888-1111	hash123	1
2	2	Ana	Lopez	Sanchez	ana@correo.com	8888-2222	hash456	2
3	3	Carlos	Perez	Mora	carlos@correo.com	8888-3333	hash789	3
4	4	Alhana	Diaz	Aguirre	alhana@correo.com	8888-4444	hash132	2
5	5	Stephanie	Brenes	Araya	brenes@correo.com	8888-5555	hash856	2
6	6	Carlos	Gutierrez	Quiros	gutierrez@correo.com	8888-6666	hash098	2
7	7	Sandra	Rojas	Brenes	sandra@correo.com	8877-1234	hash139	2
8	8	Stephanie	Brenes	Dobles	dobles@correo.com	8877-2345	hash176	2
9	9	Jose	Hernandez	Perez	hernandez@correo.com	8877-3456	hash086	2
10	10	Martha	Fonseca	Artavia	martha@correo.com	8877-4567	hash443	2
11	11	Manuel	Cruz	Cordoba	cruz@correo.com	8877-5678	hash406	2
12	12	Alejandro	Leiva	Quiros	quiros@correo.com	8877-5679	hash998	2
13	13	Christian	Marin	Hernandez	christian@correo.com	8877-6789	hash566	2
14	14	Roberto	Brenes	Araya	roberto@correo.com	8877-7890	hash444	2

Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 2 en 0,193 segundos

	ID_RESERVA	ID_USUARIO	ID_SALA	ID_ESTADO	FECHA	HORA_INICIO	HORA_FIN	COSTO_TOTAL
1	1	2	1	2	15/03/26	15/03/26 08:00:00,000000000 AM	15/03/26 10:00:00,000000000 AM	30000
2	2	2	2	1	16/03/26	16/03/26 01:00:00,000000000 PM	16/03/26 03:00:00,000000000 PM	20000

Anexo 12 Consultas de validación

```
--Probar relaciones

SELECT p.ID_Piso, p.Numero_Piso, e.Nombre_Edificio
FROM PISO p
JOIN EDIFICIO e ON p.ID_Edificio = e.ID_Edificio;


SELECT u.Nombre, u.Apellido_1, r.Descripcion_Rol
FROM USUARIOS u
JOIN ROLES r ON u.ID_Rol = r.ID_Rol;
```

Anexo 13 Consultas de validación

```sql
SELECT s.Nombre_Sala, ts.Descripcion_Tipo, p.Numero_Piso
FROM SALAS s
JOIN TIPO_SALA ts ON s.ID_Tipo = ts.ID_Tipo
JOIN PISO p ON s.ID_Piso = p.ID_Piso;


SELECT r.ID_Reserva, u.Nombre, s.Nombre_Sala, es.Nombre_Estado, r.Costo_Total
FROM RESERVAS r
JOIN USUARIOS u ON r.ID_Usuario = u.ID_Usuario
JOIN SALAS s ON r.ID_Sala = s.ID_Sala
JOIN ESTADOS es ON r.ID_Estado = es.ID_Estado;

SELECT rs.ID_Reserva, se.Nombre_Servicio, rs.Cantidad
FROM RESERVA_SERVICIOS rs
JOIN SERVICIOS se ON rs.ID_Servicio = se.ID_Servicio;


SELECT p.ID_Pago, p.ID_Reserva, p.Monto_Total, p.Metodo_Pago
FROM PAGOS p;


SELECT b.ID_Log, b.ID_Reserva, ea.Nombre_Estado AS Estado_Anterior, en.Nombre_Estado AS Estado_Nuevo, b.Fecha_Cambio
FROM BITACORA_CAMBIOS b
JOIN ESTADOS ea ON b.ID_Estado_Anterior = ea.ID_Estado
JOIN ESTADOS en ON b.ID_Estado_Nuevo = en.ID_Estado;

---Integridad referencial
```

Anexo 14 Consultas de validación

```sql
SELECT rs.ID_Reserva, se.Nombre_Servicio, rs.Cantidad
FROM RESERVA_SERVICIOS rs
JOIN SERVICIOS se ON rs.ID_Servicio = se.ID_Servicio;
```

Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 3 en 0,2 segundos

	ID_RESERVA	NOMBRE_SERVICIO	CANTIDAD
1	1	Proyector	1
2	1	Coffee Break	1
3	2	Pizarra	1

Anexo 15 Consultas de validación

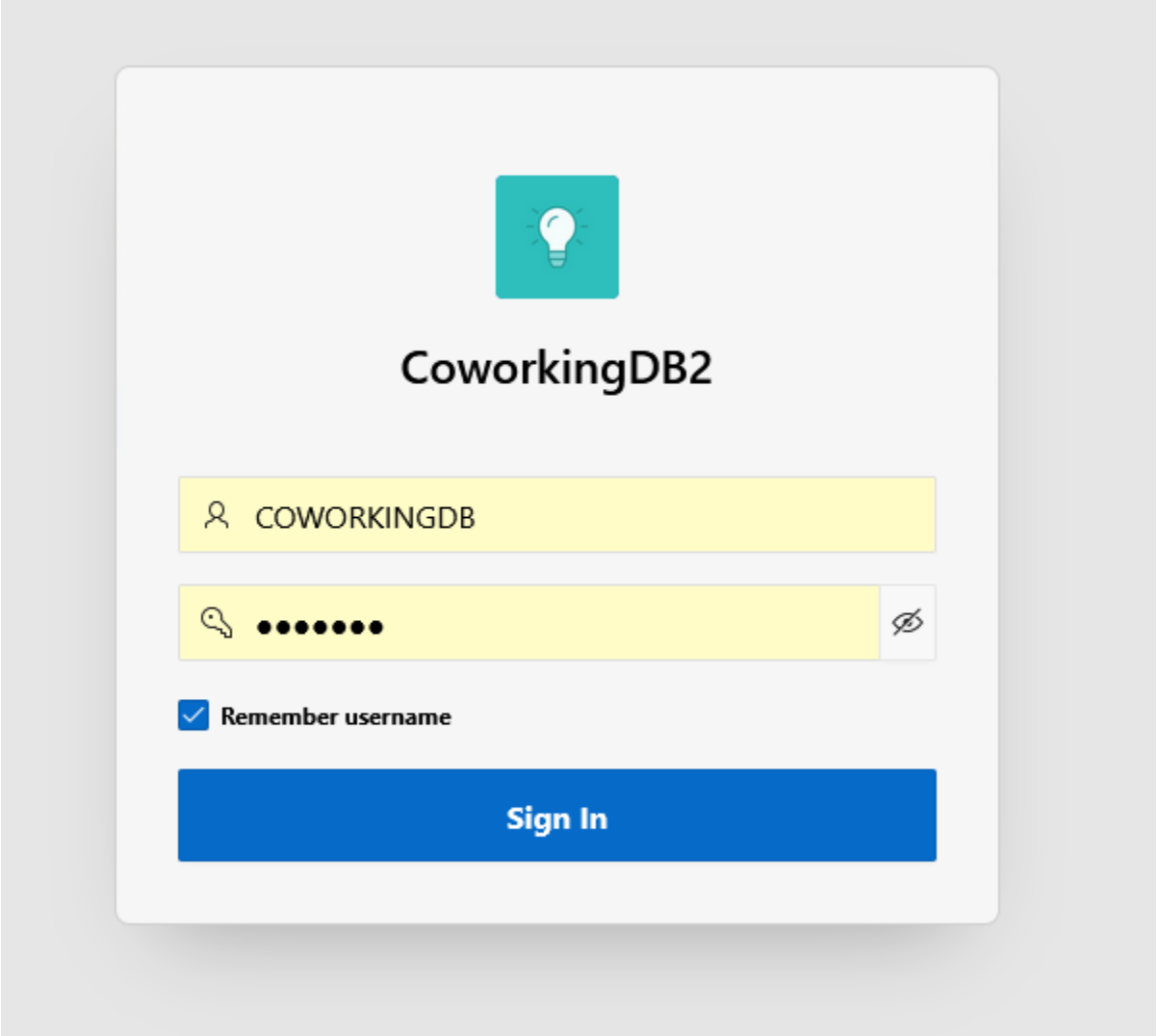
```sql
SELECT b.ID_Log, b.ID_Reserva, ea.Nombre_Estado AS Estado_Anterior, en.Nombre_Estado AS Estado_Nuevo, b.Fecha_Cambio
FROM BITACORA_CAMBIOS b
JOIN ESTADOS ea ON b.ID_Estado_Anterior = ea.ID_Estado
JOIN ESTADOS en ON b.ID_Estado_Nuevo = en.ID_Estado;

---Integridad referencial
```

Resultado de la Consulta

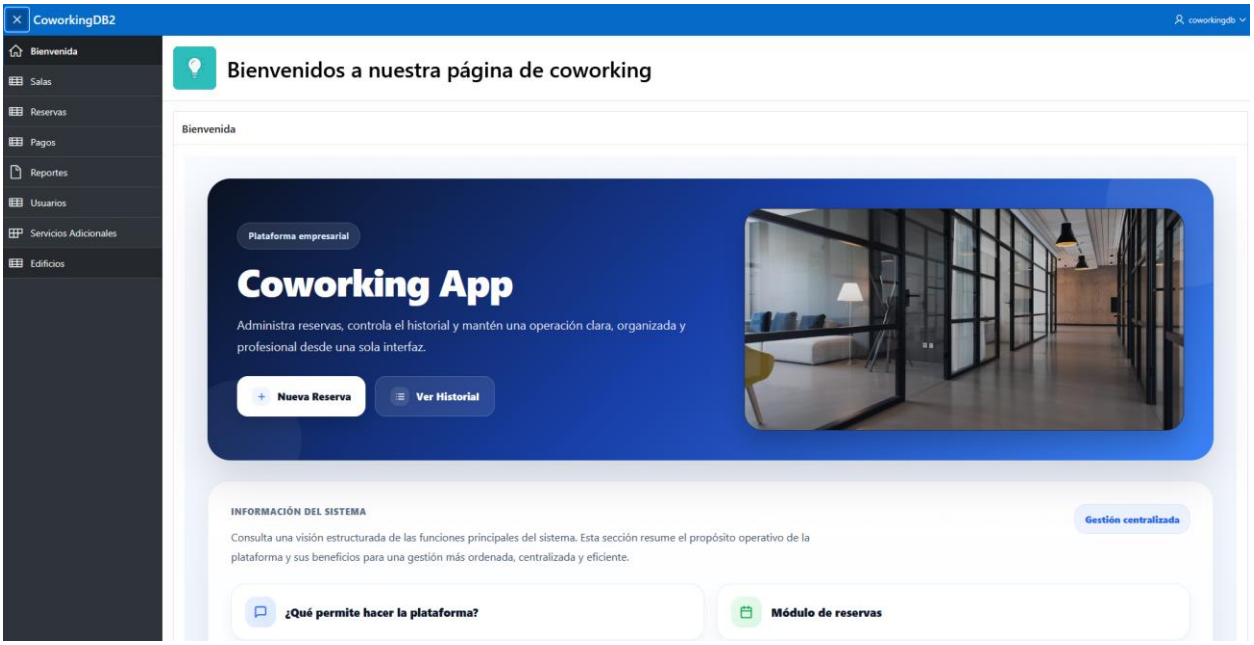
Todas las Filas Recuperadas: 3 en 0,21 segundos

	ID_LOG	ID_RESERVA	ESTADO_ANTERIOR	ESTADO_NUEVO	FECHA_CAMBIO
1	1	1	Pendiente	Confirmada	19/03/26 11:49:11,727441000 AM
2	2	1	Pendiente	Confirmada	19/03/26 11:54:14,555355000 AM
3	3	2	Pendiente	Cancelada	19/03/26 11:54:18,332432000 AM

Anexo 16 Pagina de login



Anexo 17 Pagina del dashboard



Anexo 18 Pagina de salas

Editar Sala	Nombre Sala	Capacidad	Precio Hora
	Sala Milan	6	12,000
	Sala Madrid	10	18,000
	Sala Lima	20	25,000
	Sala Francia	10	10,000
	Sala Valencia	7	6,000
	Sala Estocolmo	15	23,000
	Sala Bogota	4	7,000
	Sala Los Angeles	25	40,000

1 - 8

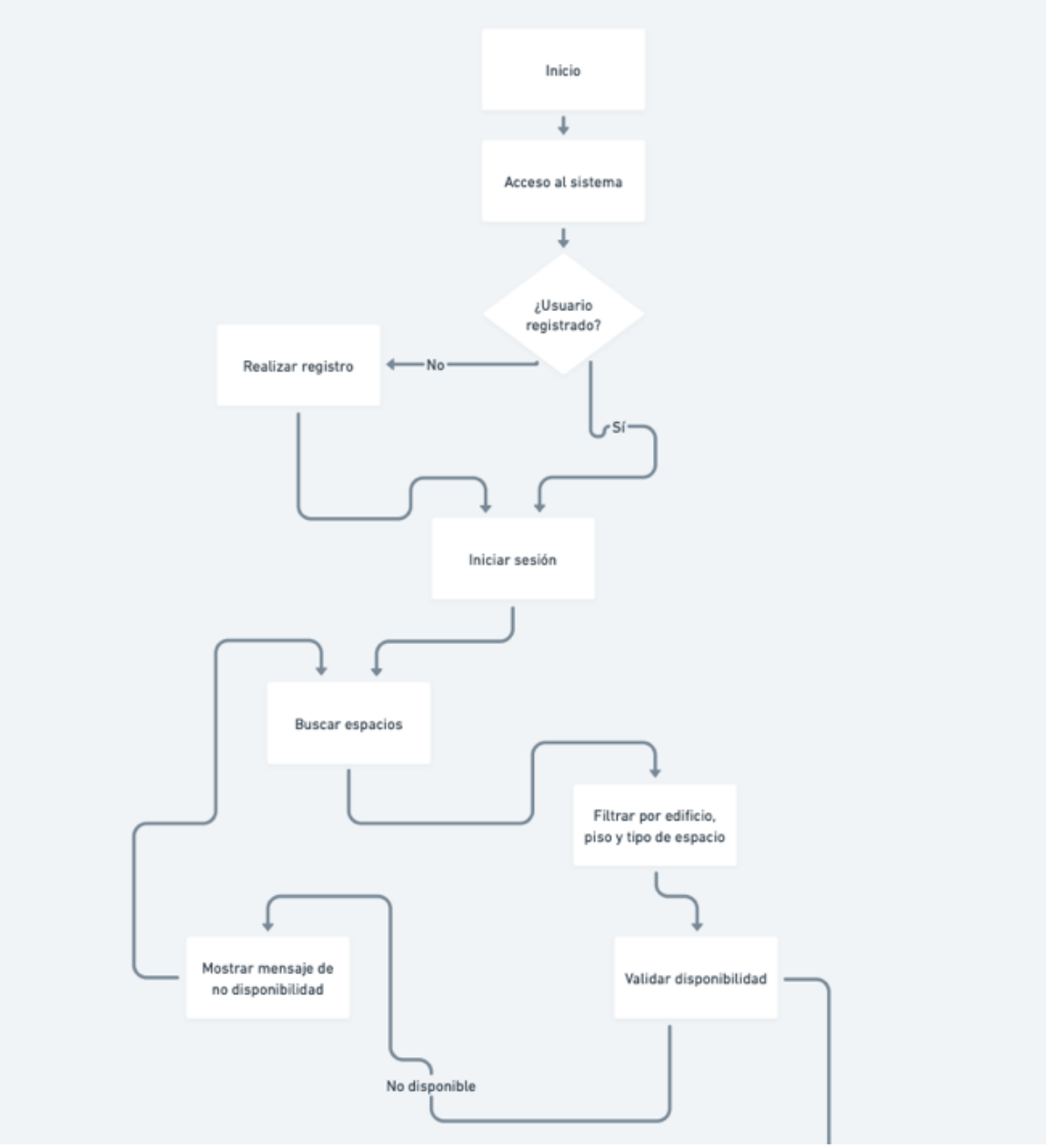
Anexo 19 Pagina de reservacion

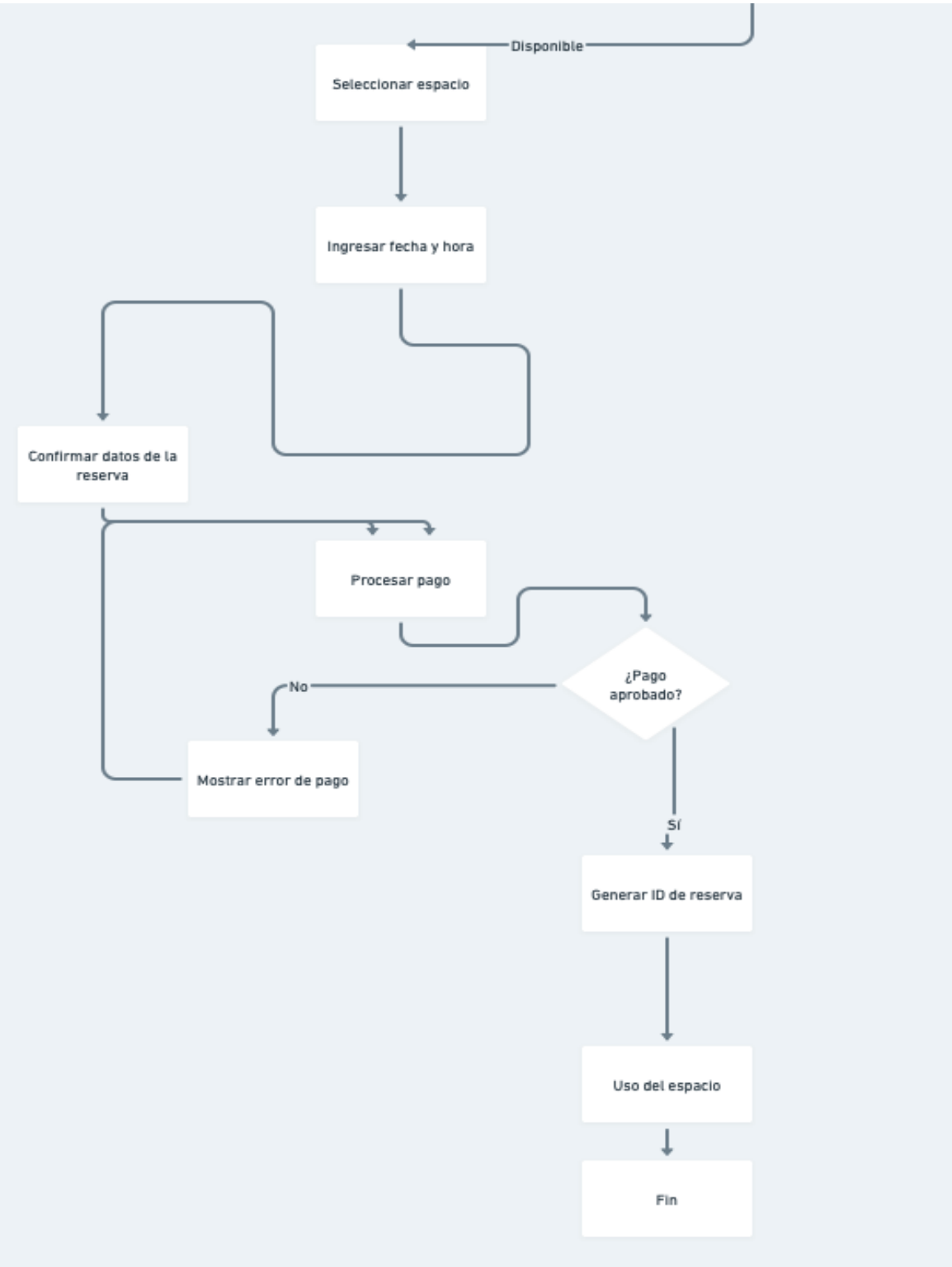
Anexo 20 Lista de reservas

Go | Actions

Id Reserva	Usuario	Nombre Sala	Nombre Estado	Fecha	Hora Inicio	Hora Fin	Costo Total	Servicios
1	Flor Flores	Sala Milan	Confirmada	20/04/2027	08:00	10:00	24000	Coffee Break (x2), Proyector (x1)
6	Ana Solis	Sala Estocolmo	Pendiente	23/04/2026	21:00	22:00	23000	Sin servicios
11	Carlos Ramirez	Sala Francia	Pendiente	30/04/2026	07:15	08:15	10000	Sin servicios
12	Carlos Ramirez	Sala Estocolmo	Pendiente	29/04/2026	10:15	12:15	46000	Sin servicios
13	Jeffrey Pasos	Sala Madrid	Confirmada	04/05/2026	07:30	08:30	18000	Sin servicios
14	Raquel Caizas	Sala Los Angeles	Pendiente	12/05/2026	09:45	11:45	80000	Sin servicios
16	Natalie Pinar	Sala Milan	Pendiente	12/05/2026	09:45	11:45	24000	Sin servicios
18	Ana Solis	Sala Lima	Pendiente	30/04/2026	12:15	15:15	75000	Sin servicios
19	Maria Lopez	Sala Estocolmo	Confirmada	29/04/2026	13:45	16:45	69000	Sin servicios
20	Maria Lopez	Sala Lima	Pendiente	13/05/2026	13:45	16:45	75000	Sin servicios
21	Daniel Quesada	Sala Lima	Confirmada	27/05/2026	13:45	16:45	75000	Sin servicios
23	Jeffrey Pasos	Sala Francia	Pendiente	27/04/2026	00:15	02:15	20000	Sin servicios
24	Flor Flores	Sala Lima	Confirmada	27/05/2026	09:15	12:15	75000	Sin servicios
25	Natalie Pinar	Sala Los Angeles	Pendiente	28/04/2026	05:30	10:30	200000	Sin servicios
26	Ana Solis	Sala Bogota	Confirmada	26/04/2026	04:45	13:45	63000	Sin servicios
27	Adrian Avilan	Sala Francia	Confirmada	07/05/2026	01:00	08:00	70000	Sin servicios
29	Adrian Avilan	Sala Bogota	Confirmada	30/04/2026	01:00	03:00	14000	Sin servicios
30	Adrian Avilan	Sala Francia	Confirmada	29/04/2026	10:00	12:00	20000	Sin servicios
35	Jefry Goleath	Sala Bogota	Confirmada	24/04/2026	11:00	12:00	7000	Sin servicios
39	Jefry Goleath	Sala Estocolmo	Confirmada	24/04/2026	15:30	17:30	46000	Sin servicios

Anexo 21


Inicio
Acceso al sistema
¿Usuario registrado?
No
Realizar registro
Sí
Iniciar sesión
Buscar espacios
Filtrar por edificio, piso y tipo de espacio
Mostrar mensaje de no disponibilidad
Validar disponibilidad
No disponible



Anexo 22 Diccionario de datos

BITACORA_CAMBIOS

Columns | Constraints | Grants | Statistics | Triggers | Dependencies | Details | Partitions | Indexes

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
ID_LOG	NUMBER	No	"JGOLEATH"."ISEQ$$_141984".nextval	1	null
ID_RESERVA	NUMBER	No	null	2	null
ID_ESTADO_ANTERIOR	NUMBER	Yes	null	3	null
ID_ESTADO_NUEVO	NUMBER	Yes	null	4	null
FECHA_CAMBIO	TIMESTAMP(6)	Yes	CURRENT_TIMESTAMP	5	null

Anexo 23 Diccionario de datos

EDIFICIO

Columns | Constraints | Grants | Statistics | Triggers | Dependencies | Details | Partitions | Indexes

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
ID_EDIFICIO	NUMBER	No	"JGOLEATH"."ISEQ$$_141942".nextval	1	null
NOMBRE_EDIFICIO	VARCHAR2(100 BYTE)	No	null	2	null
DIRECCION	VARCHAR2(255 BYTE)	No	null	3	null

Anexo 24 Diccionario de datos

ESTADOS

Columns | Constraints | Grants | Statistics | Triggers | Dependencies | Details | Partitions | Indexes

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
ID_ESTADO	NUMBER	No	"JGOLEATH"."ISEQ$$_141966".nextval	1	null
NOMBRE_ESTADO	VARCHAR2(50 BYTE)	No	null	2	null

Anexo 25 Diccionario de datos

PAGOS

Columns | Constraints | Grants | Statistics | Triggers | Dependencies | Details | Partitions | Indexes

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
ID_PAGO	NUMBER	No	"JGOLEATH"."ISEQ$$_141979".nextval	1	null
ID_RESERVA	NUMBER	No	null	2	null
MONTO_TOTAL	NUMBER(10,2)	No	null	3	null
FECHA_PAGO	TIMESTAMP(6)	Yes	CURRENT_TIMESTAMP	4	null
METODO_PAGO	VARCHAR2(50 BYTE)	No	null	5	null
ESTADO_PAGO	VARCHAR2(20 BYTE)	No	'Pendiente'	6	null
REFERENCIA_TRANSACCION	VARCHAR2(100 BYTE)	Yes	null	7	null
REGISTRADO_POR	NUMBER	Yes	null	8	null

Anexo 26 Diccionario de datos

PISO

Columns | Constraints | Grants | Statistics | Triggers | Dependencies | Details | Partitions | Indexes

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
ID_PISO	NUMBER	No	"JGOLEATH"."ISEQ$$_141945".nextval	1	null
NUMERO_PISO	NUMBER(2,0)	No	null	2	null
ID_EDIFICIO	NUMBER	No	null	3	null

Anexo 27 Diccionario de datos

RESERVAS

Columns | Constraints | Grants | Statistics | Triggers | Dependencies | Details | Partitions | Indexes

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
ID_RESERVA	NUMBER	No	"JGOLEATH"."ISEQ$$_141970".nextval	1	null
ID_USUARIO	NUMBER	No	null	2	null
ID_SALA	NUMBER	No	null	3	null
ID_ESTADO	NUMBER	No	null	4	null
FECHA	DATE	No	null	5	null
HORA_INICIO	TIMESTAMP(6)	No	null	6	null
HORA_FIN	TIMESTAMP(6)	No	null	7	null
COSTO_TOTAL	NUMBER(10,2)	Yes	null	8	null

Anexo28 Diccionario de datos

RESERVA_SERVICIOS

Columns | Constraints | Grants | Statistics | Triggers | Dependencies | Details | Partitions | Indexes

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
ID_RESERVA	NUMBER	No	null	1	null
ID_SERVICIO	NUMBER	No	null	2	null
CANTIDAD	NUMBER(2,0)	No	1	3	null

Anexo 29 Diccionario de datos

ROLES

Columns | Constraints | Grants | Statistics | Triggers | Dependencies | Details | Partitions | Indexes

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
ID_ROL	NUMBER	No	"JGOLEATH"."ISEQ$$_141949".nextval	1	null
DESCRIPCION_ROL	VARCHAR2(50 BYTE)	No	null	2	null

Anexo 30 Diccionario de datos

SALAS

Columns | Constraints | Grants | Statistics | Triggers | Dependencies | Details | Partitions | Indexes

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
ID_SALA	NUMBER	No	"JGOLEATH"."ISEQ$$_141962".nextval	1	null
NOMBRE_SALA	VARCHAR2(100 BYTE)	No	null	2	null
CAPACIDAD	NUMBER(3,0)	No	null	3	null
PRECIO_HORA	NUMBER(8,2)	No	null	4	null
ID_TIPO	NUMBER	No	null	5	null
ID_PISO	NUMBER	No	null	6	null

Anexo 31 Diccionario de datos

SERVICIOS

Columns | Constraints | Grants | Statistics | Triggers | Dependencies | Details | Partitions | Indexes

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
ID_SERVICIO	NUMBER	No	"JGOLEATH"."ISEQ$$_141973".nextval	1	null
NOMBRE_SERVICIO	VARCHAR2(100 BYTE)	No	null	2	null
PRECIO_ADICIONAL	NUMBER(8,2)	No	null	3	null

Anexo 32 Diccionario de datos

TIPO_SALA

Columns | Constraints | Grants | Statistics | Triggers | Dependencies | Details | Partitions | Indexes

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
ID_TIPO	NUMBER	No	"JGOLEATH"."ISEQ$$_141958".nextval	1	null
DESCRIPCION_TIPO	VARCHAR2(50 BYTE)	No	null	2	null

Anexo 33 Diccionario de datos

USUARIOS

Columns | Constraints | Grants | Statistics | Triggers | Dependencies | Details | Partitions | Indexes

COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
ID_USUARIO	NUMBER	No	"JGOLEATH"."ISEQ$$_141953".nextval	1	null
NOMBRE	VARCHAR2(50 BYTE)	No	null	2	null
APELLIDO_1	VARCHAR2(50 BYTE)	No	null	3	null
APELLIDO_2	VARCHAR2(50 BYTE)	Yes	null	4	null
CORREO	VARCHAR2(100 BYTE)	No	null	5	null
TELEFONO	VARCHAR2(20 BYTE)	No	null	6	null
CODIGO_PAIS	VARCHAR2(5 BYTE)	No	'CR'	7	null
CONTRASENA_HASH	VARCHAR2(255 BYTE)	No	null	8	null
ID_ROL	NUMBER	No	null	9	null

Anexo 34 Diccionario de datos

Tabla	Columna	Tipo de Dato	Descripción
EDIFICIO	id_edificio	NUMBER (PK)	Identificador único de la sede.
	nombre_edificio	VARCHAR2(100)	Nombre de la ubicación.
	direccion	VARCHAR2(255)	Dirección física completa.
PISO	id_piso	NUMBER (PK)	Identificador del nivel del edificio.
	numero_piso	NUMBER(2)	Número de planta (0-99).
	id_edificio	NUMBER (FK)	Relación con la tabla EDIFICIO.
ROLES	id_rol	NUMBER (PK)	ID del perfil de usuario.
	descripcion_rol	VARCHAR2(50)	Nombre del rol (Admin, Cliente, etc.).
USUARIOS	id_usuario	NUMBER (PK)	Identificador único del usuario.
	nombre	VARCHAR2(50)	Nombre(s) del usuario.
	apellido_1	VARCHAR2(50)	Primer apellido.
	correo	VARCHAR2(100)	Email único (Validado con REGEXP)
	telefono	VARCHAR2(20)	Número telefónico (8 dígitos)
	id_rol	NUMBER (FK)	Perfil asignado al usuario.
TIPO_SALA	id_tipo	NUMBER (PK)	ID de categoría de sala.
	descripcion_tipo	VARCHAR2(50)	Ej: Sala de Reunión, Privada.
SALAS	id_sala	NUMBER (PK)	Identificador de la sala física.
	nombre_sala	VARCHAR2(100)	Nombre único de la sala.
	capacidad	NUMBER(3)	Aforo máximo permitido.
	precio_hora	NUMBER(8,2)	Costo base por hora de uso.
	id_piso	NUMBER (FK)	Ubicación física (piso) de la sala.
ESTADOS	id_estado	NUMBER (PK)	ID del estado de la reserva.
	nombre_estado	VARCHAR2(50)	Pendiente, Confirmada, Cancelada.
RESERVAS	id_reserva	NUMBER (PK)	Identificador de la transacción.
	fecha	DATE	Fecha programada para el evento.
	hora_inicio	TIMESTAMP	Tiempo exacto de inicio.
	hora_fin	TIMESTAMP	Tiempo exacto de finalización.
	costo_total	NUMBER(10,2)	Monto calculado (Trigger trg_reservas_calcular_costo)
SERVICIOS	id_servicio	NUMBER (PK)	ID del servicio adicional.
	nombre_servicio	VARCHAR2(100)	Ej: Proyector, Catering.
	precio_adicional	NUMBER(8,2)	Costo por unidad del servicio.
RESERVA_SERVICIOS	id_reserva	NUMBER (PK, FK)	Asociación con la reserva.
	id_servicio	NUMBER (PK, FK)	Asociación con el servicio.
	cantidad	NUMBER(2)	Cantidad solicitada
PAGOS	id_pago	NUMBER (PK)	ID del comprobante de pago.
	monto_total	NUMBER(10,2)	Total (Reserva + Servicios).
	metodo_pago	VARCHAR2(50)	Tarjeta, SINPE, etc. (Validado REGEXP)
BITACORA_CAMBIOS	id_log	NUMBER (PK)	ID del evento de auditoría.
	id_reserva	NUMBER (FK)	Reserva que sufrió el cambio.
	id_estado_nuevo	NUMBER (FK)	Nuevo estado asignado.